

(54) **SAMPLE DISTANCE CALCULATION FOR GEOMETRIC PARTITION MODE**

(71) Applicant: **HUAWEI TECHNOLOGIES CO., LTD.**, Shenzhen (CN)

(72) Inventors: **Semih ESENLIK**, Munich (DE); **Max BLAESER**, Munich (DE); **Han GAO**, Shenzhen (CN); **Zhijie ZHAO**, Shenzhen (CN); **Anand Meher KOTRA**, Munich (DE); **Biao WANG**, Shenzhen (CN); **Elena Alexandrovna ALSHINA**, Munich (DE)

(21) Appl. No.: **19/206,310**

(22) Filed: **May 13, 2025**

Related U.S. Application Data

(63) Continuation of application No. 17/369,862, filed on Jul. 7, 2021, now Pat. No. 12,323,588, which is a continuation of application No. PCT/CN2020/097628, filed on Jun. 23, 2020.

Foreign Application Priority Data

Jun. 24, 2019 (WO) PCT/EP2019/066685

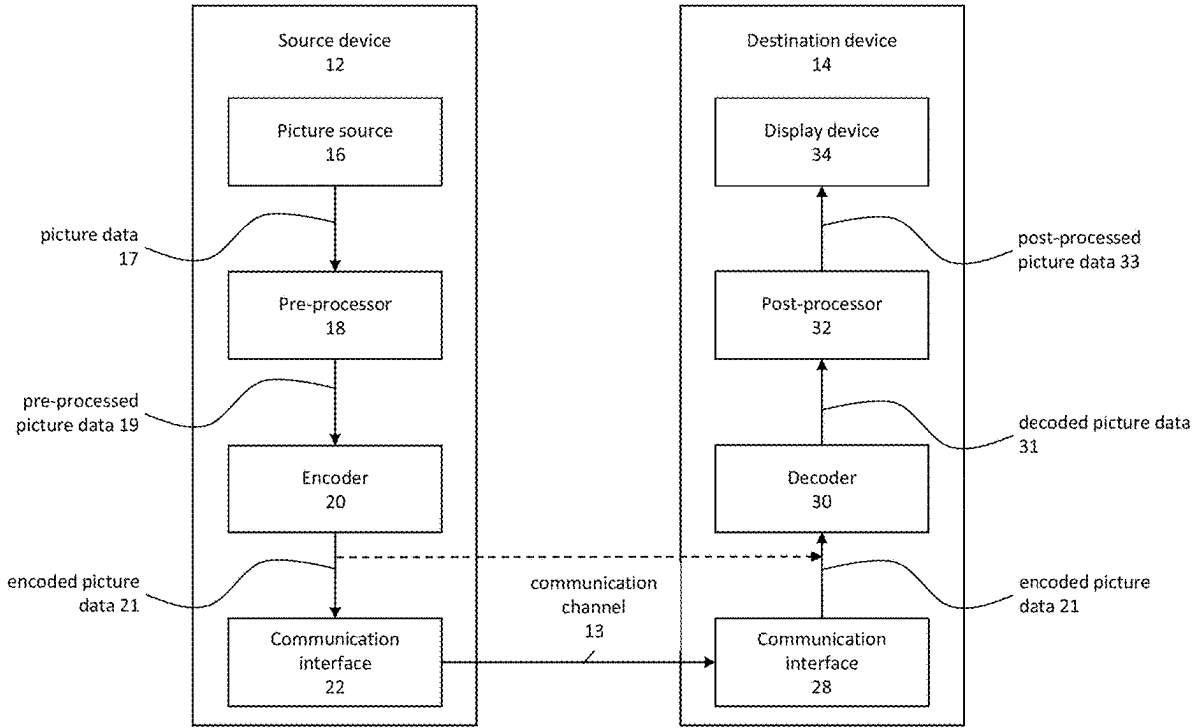
Publication Classification

(51) **Int. Cl.**
H04N 19/119 (2014.01)
H04N 19/105 (2014.01)
H04N 19/11 (2014.01)
H04N 19/132 (2014.01)
H04N 19/159 (2014.01)
H04N 19/176 (2014.01)
H04N 19/50 (2014.01)

(52) **U.S. Cl.**
CPC *H04N 19/119* (2014.11); *H04N 19/176* (2014.11); *H04N 19/50* (2014.11); *H04N 19/105* (2014.11); *H04N 19/11* (2014.11); *H04N 19/132* (2014.11); *H04N 19/159* (2014.11)

(57) **ABSTRACT**

A method of coding implemented by a decoding device or encoding device, the method comprising obtaining a value of an angle parameter for a current block; obtaining a value of a width of the current block and a value of a height of the current block; calculating a ratio value between the value of the width and the value of height; obtaining a first value according to the value of angle parameter and the ratio value; calculating a sample distance value for a sample in the current block according to the first value; obtaining a prediction value for the sample in the current block according to the sample distance value.



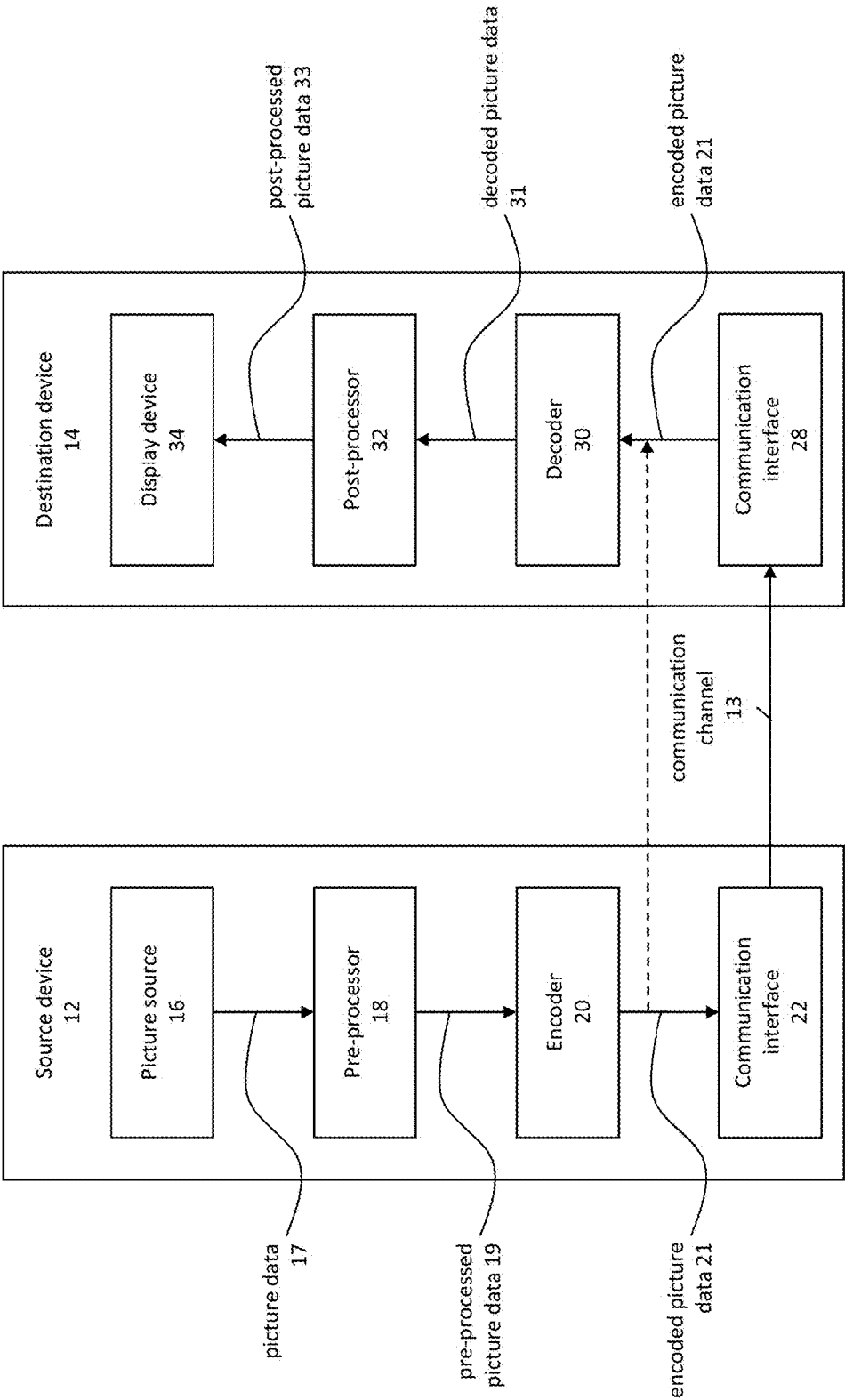


FIG. 1A

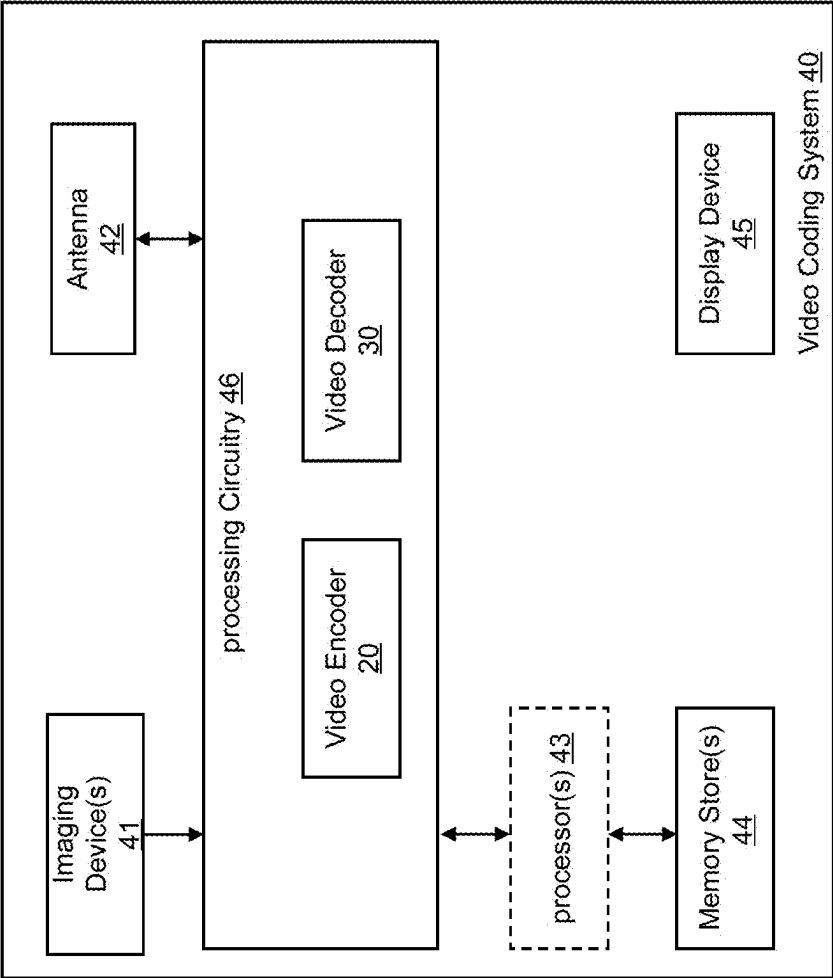


FIG. 1B

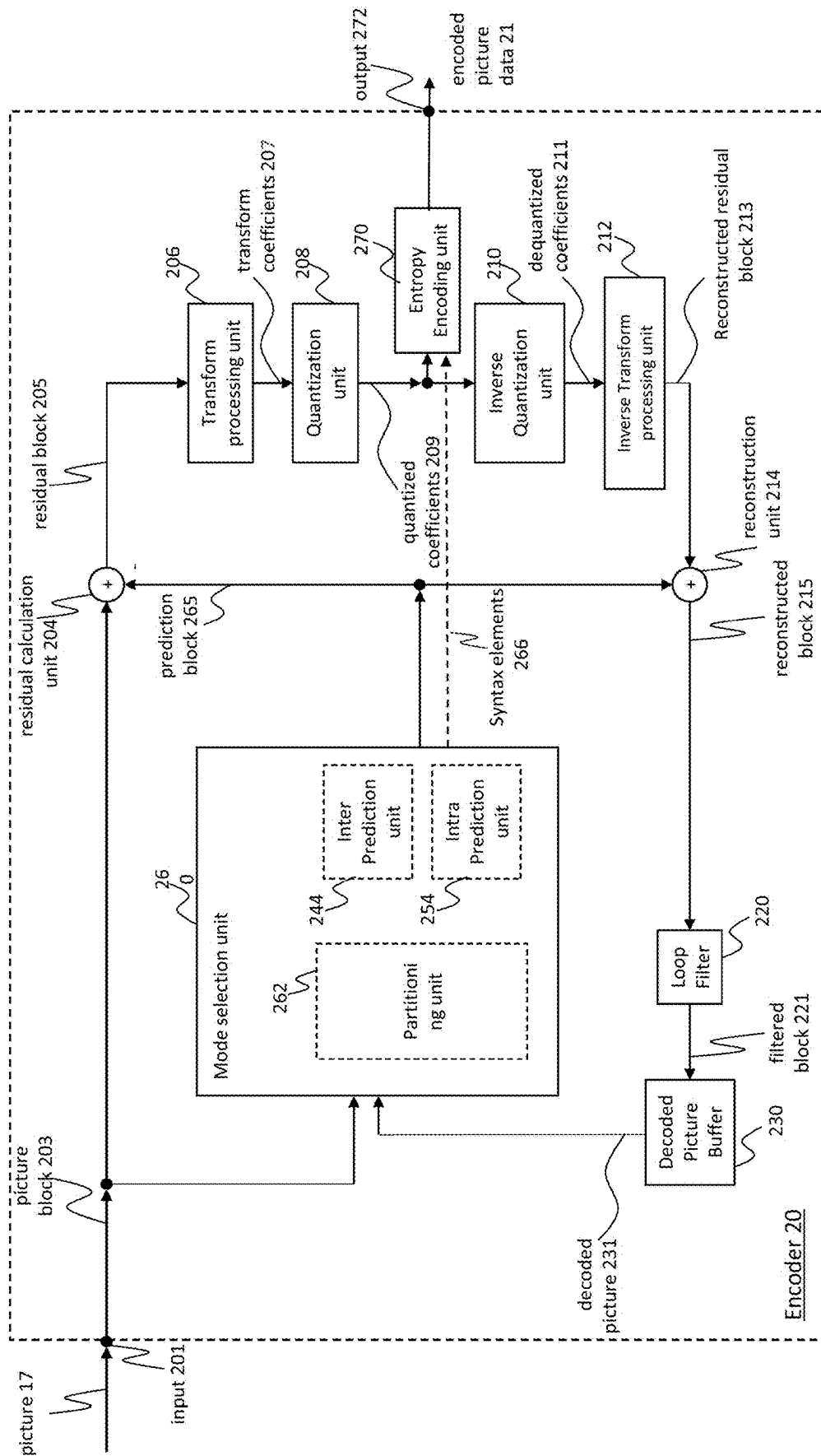


FIG. 2

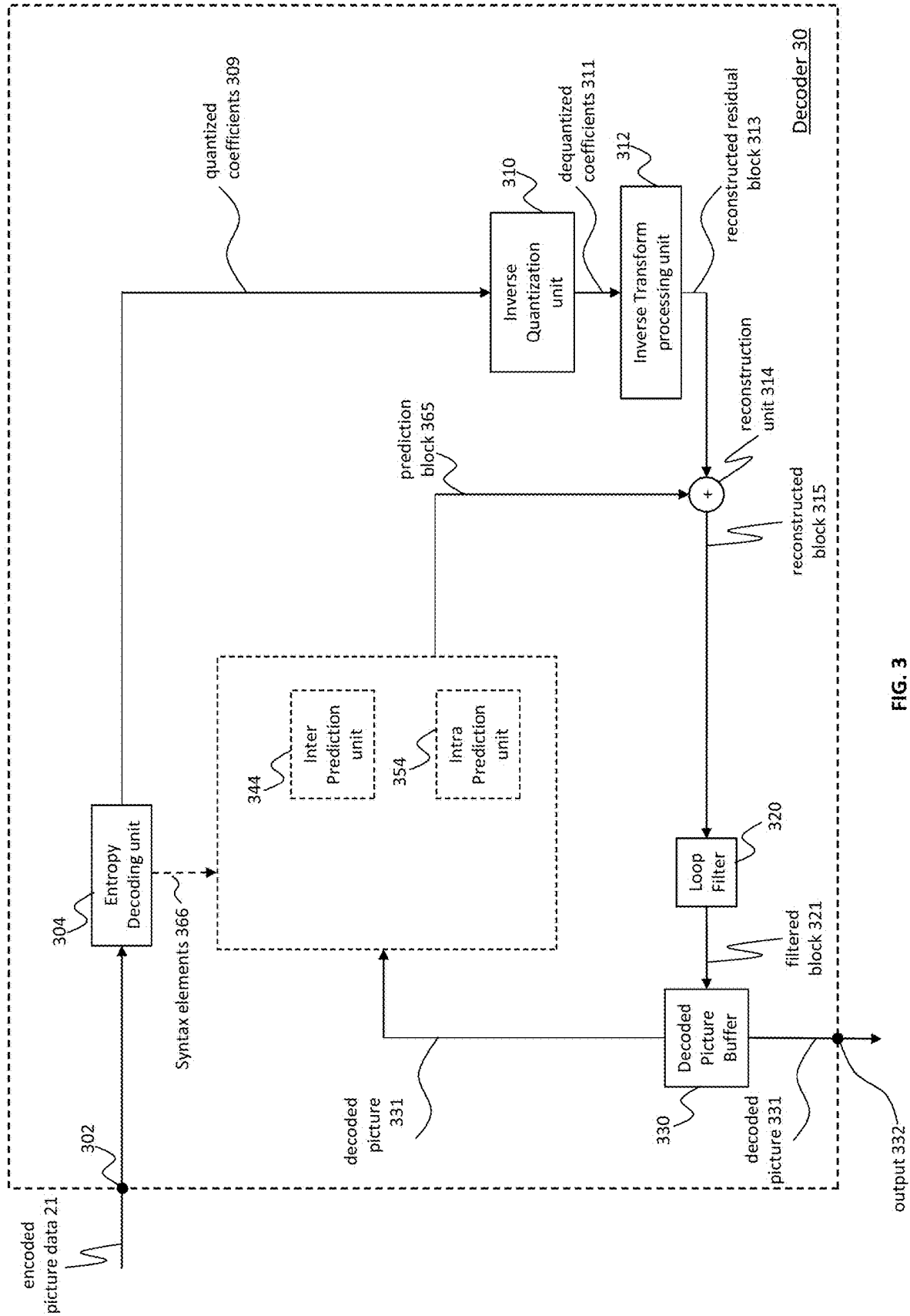


FIG. 3

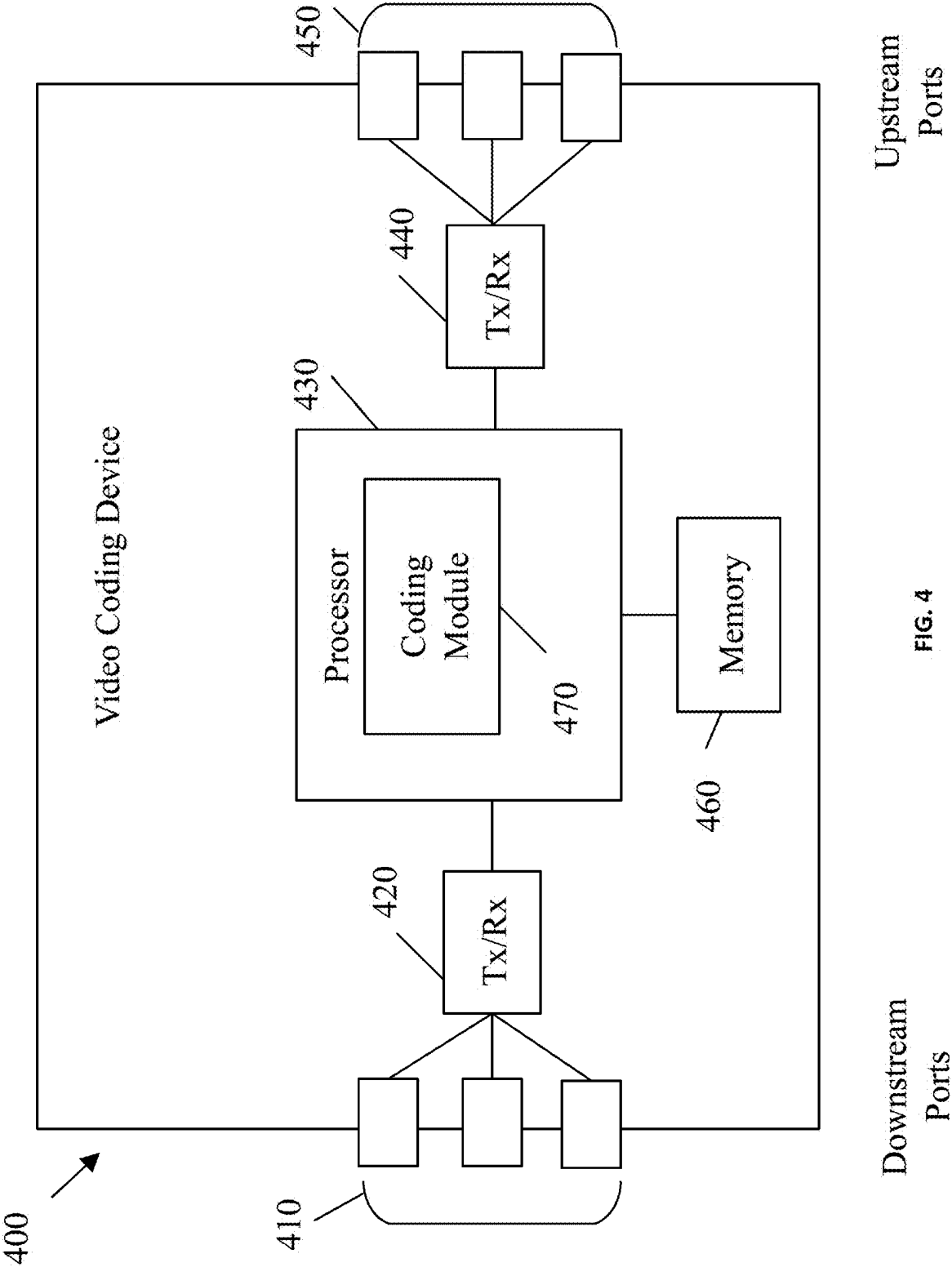


FIG. 4

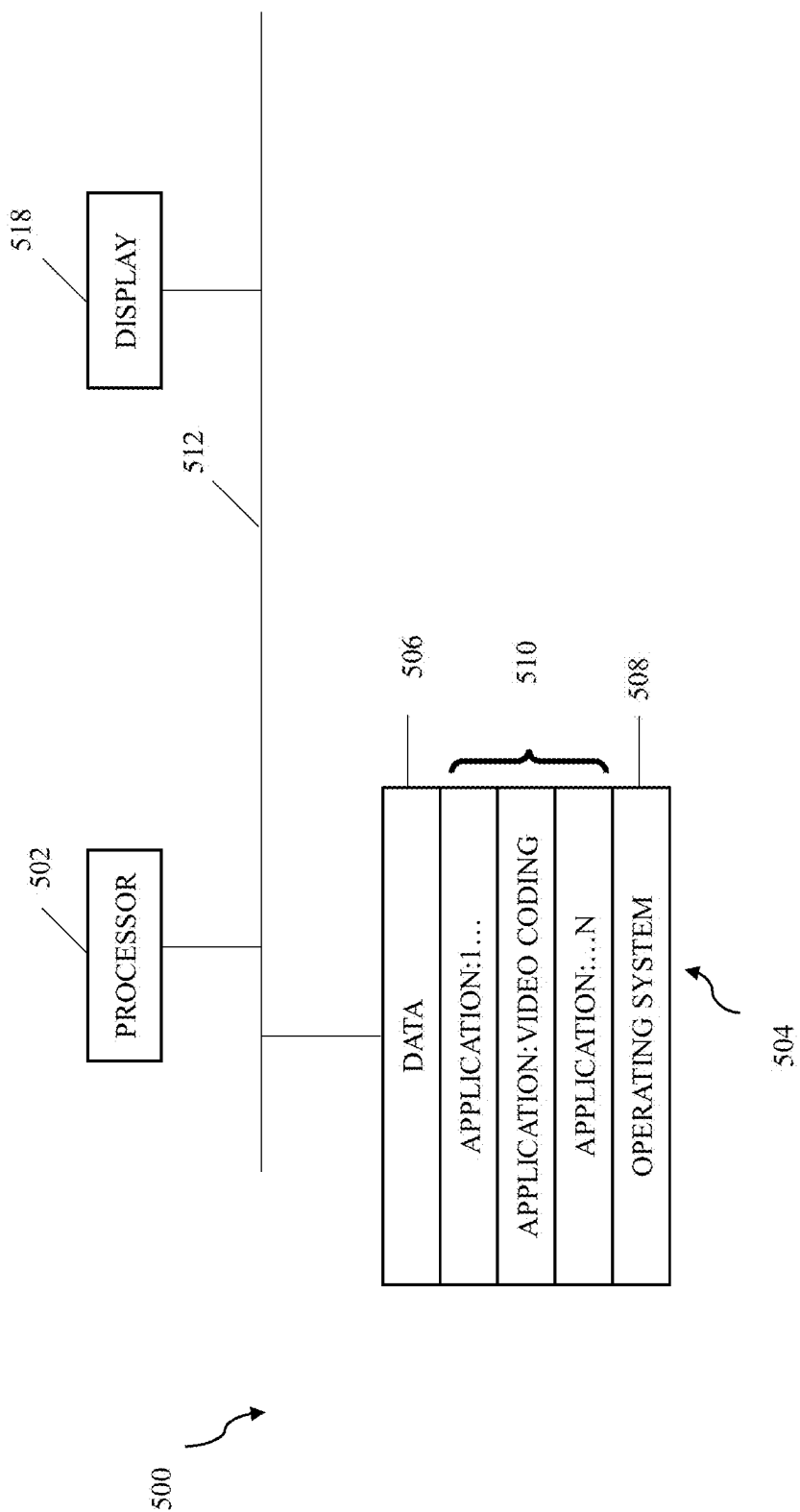


FIG. 5

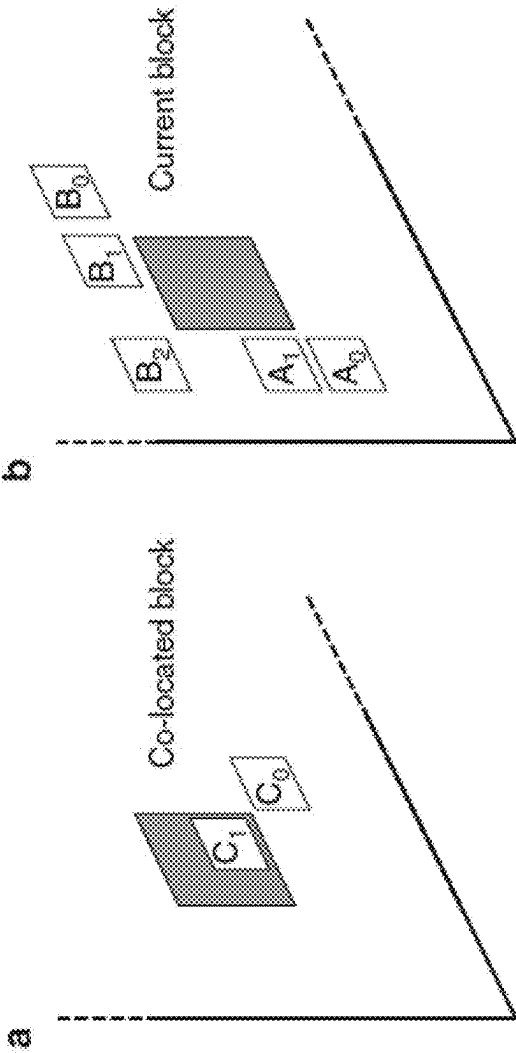
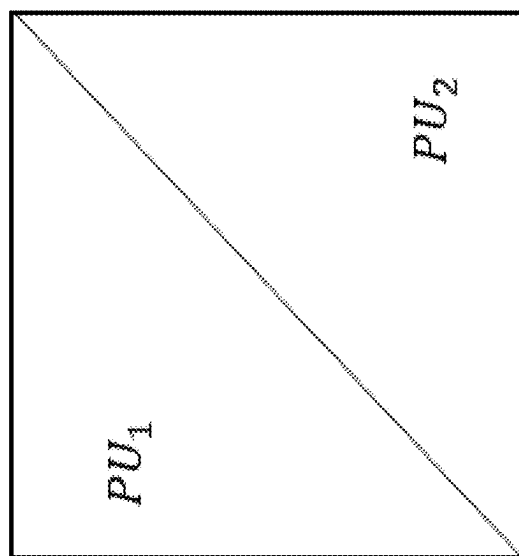
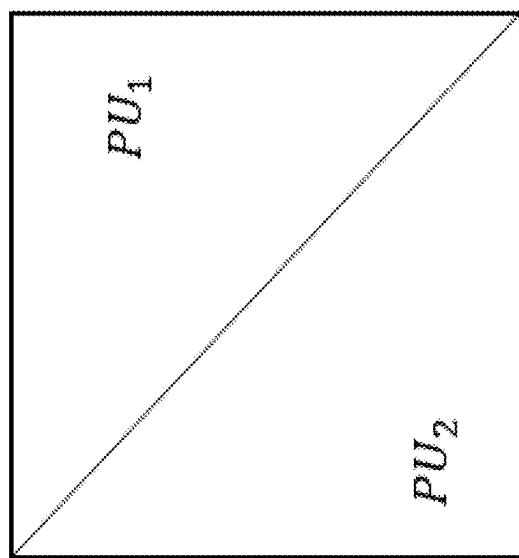


FIG. 6b

FIG. 6a



splitting from
top-right corner to
bottom-left corner



splitting from
top-left corner to
bottom-right corner

FIG. 7

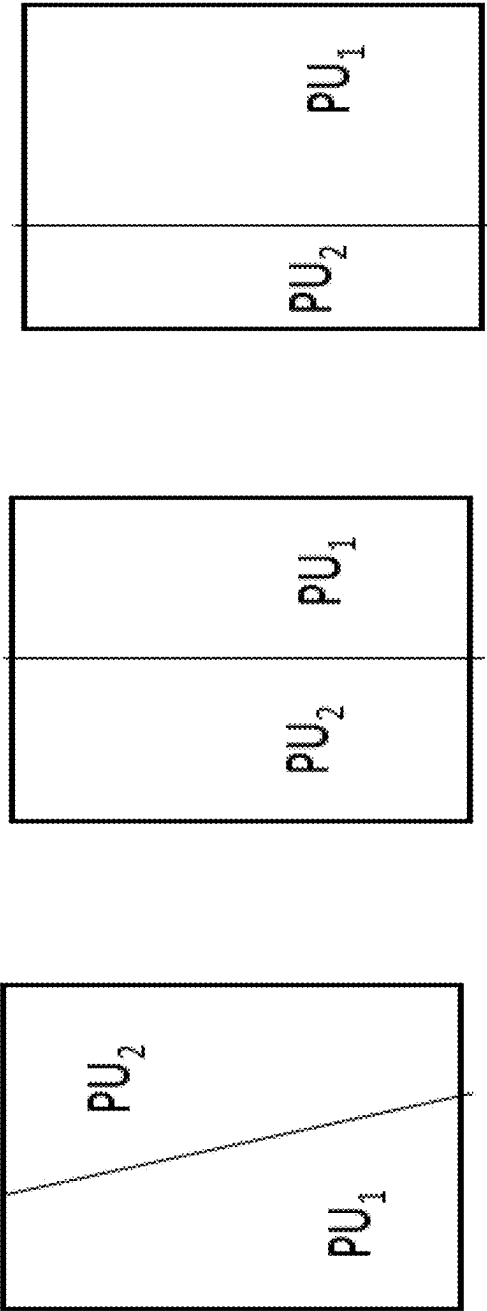


FIG. 8

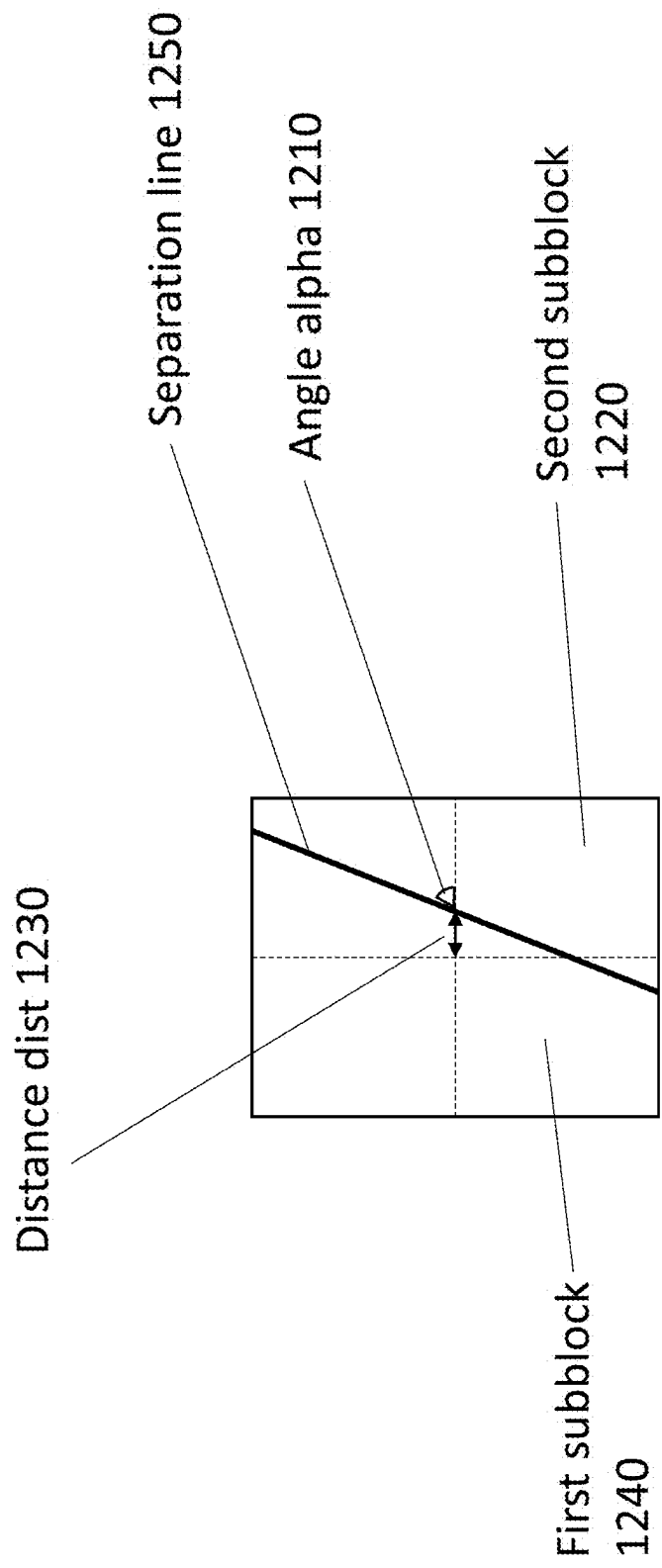


FIG. 9

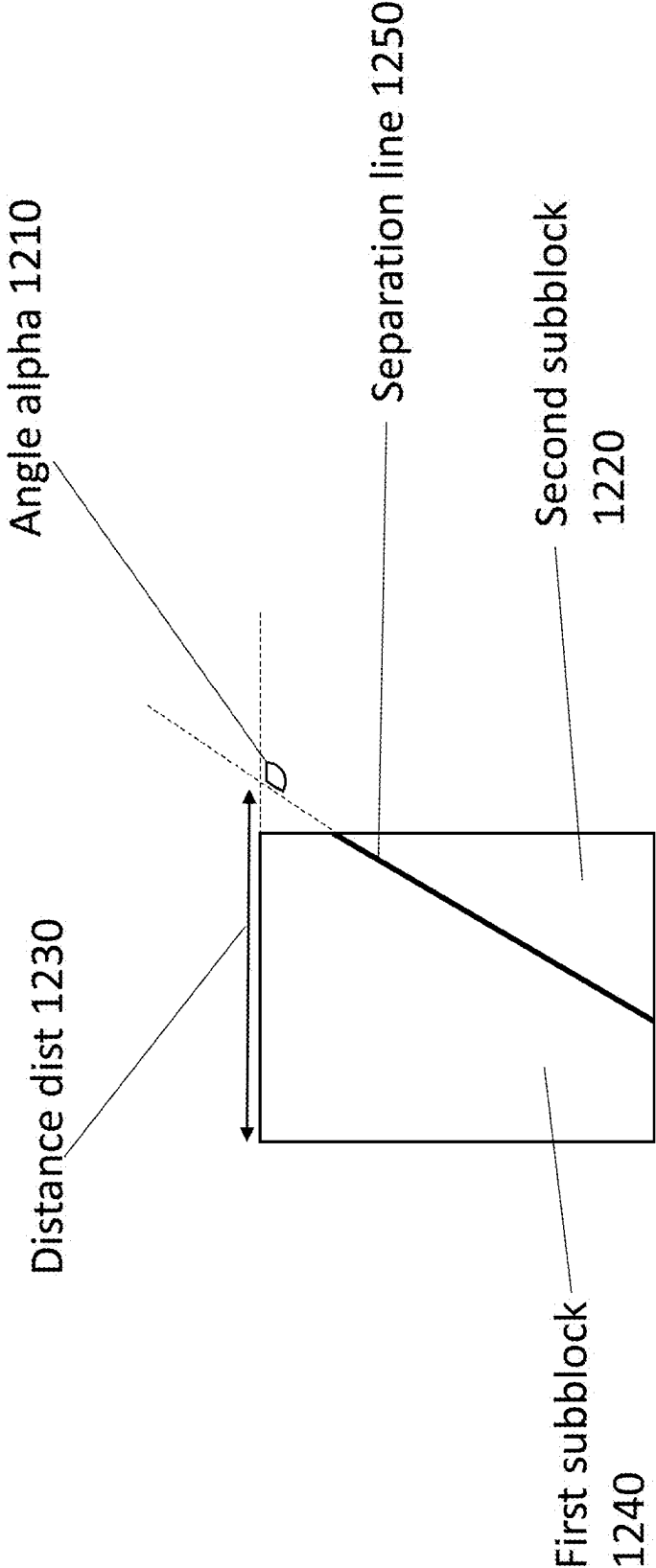


FIG. 10

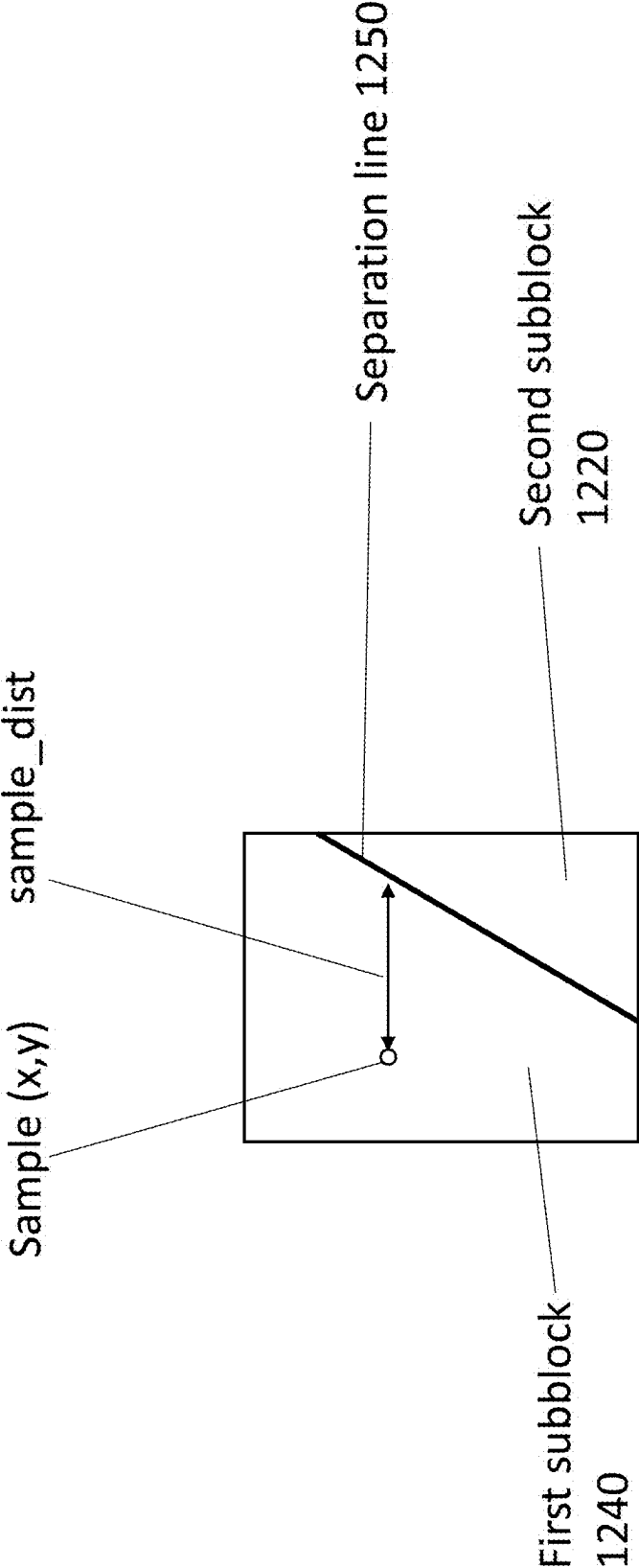


FIG. 11

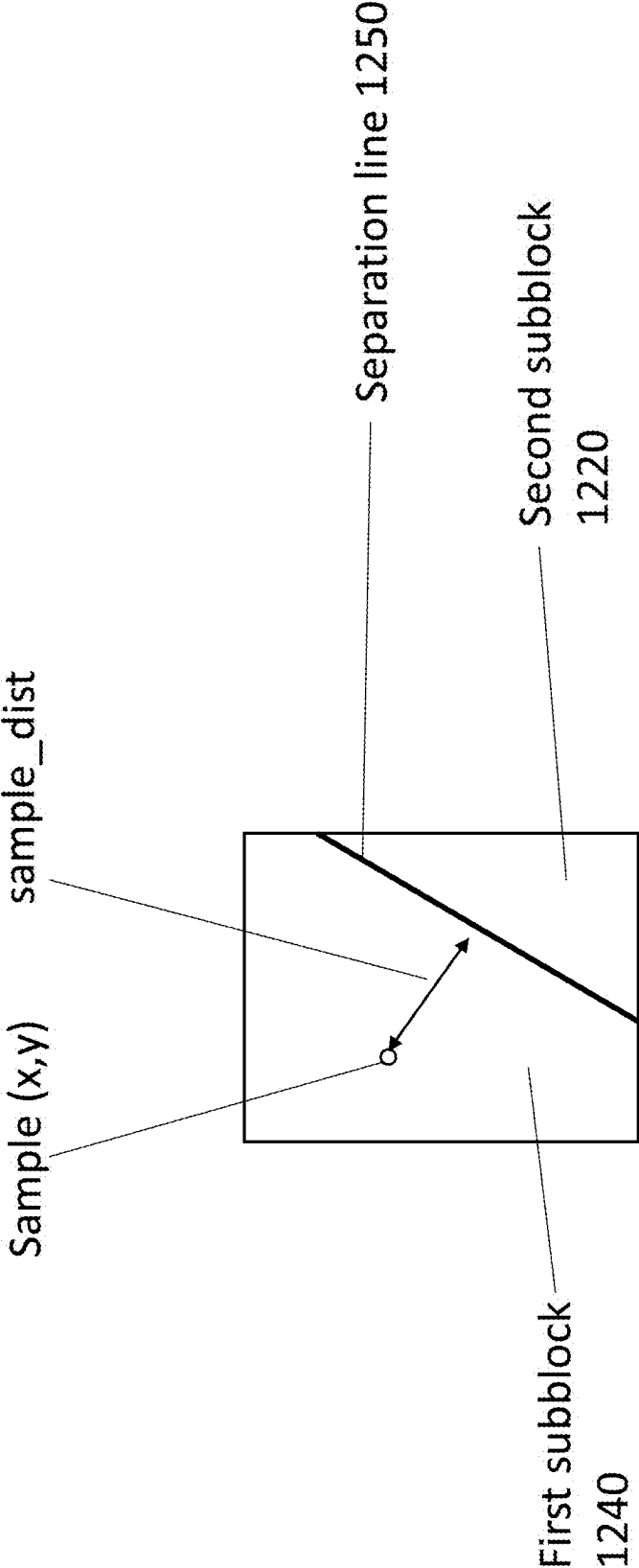


FIG. 12

whRatio	alphaN	stepD
3	0	0
3	1	158
3	2	306
3	3	438
3	4	549
3	5	636
3	6	694
3	7	721
3	8	717
4	0	0
4	1	318
4	2	619
4	3	893
4	4	1129
4	5	1317
4	6	1450
4	7	1524
4	8	1536

whRatio	alphaN	stepD
0	0	0
0	1	18
0	2	31
0	3	40
0	4	42
0	5	40
0	6	31
0	7	18
0	8	0
1	0	0
1	1	38
1	2	71
1	3	97
1	4	115
1	5	125
1	6	126
1	7	118
1	8	102
2	0	0
2	1	78
2	2	149
2	3	210
2	4	260
2	5	295
2	6	315
2	7	319
2	8	307

FIG. 13

idx	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
f[idx]	64	63	59	53	45	36	24	12	0	-12	-24	-36	-45	-53	-59	-63
idx	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
f[idx]	-64	-63	-59	-53	-45	-36	-24	-12	0	12	24	36	45	53	59	63

FIG. 14

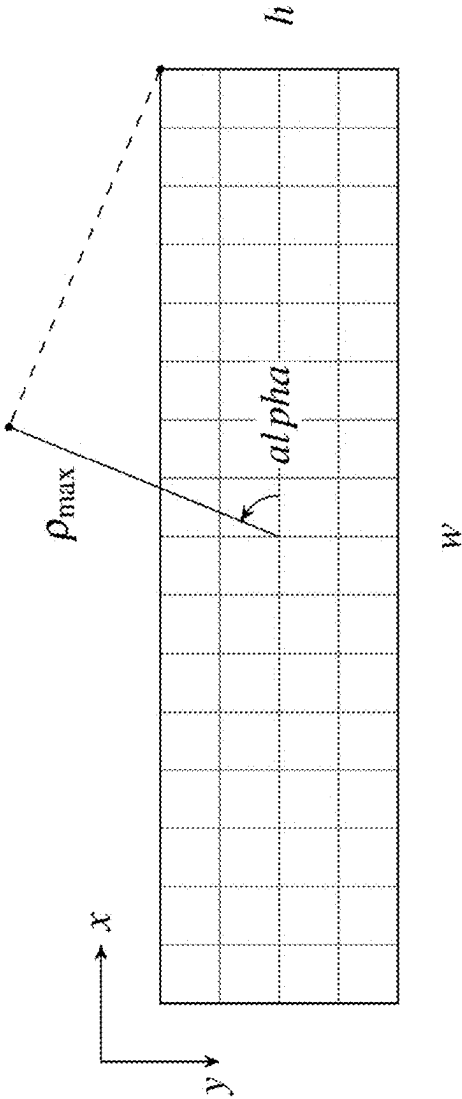


FIG. 15

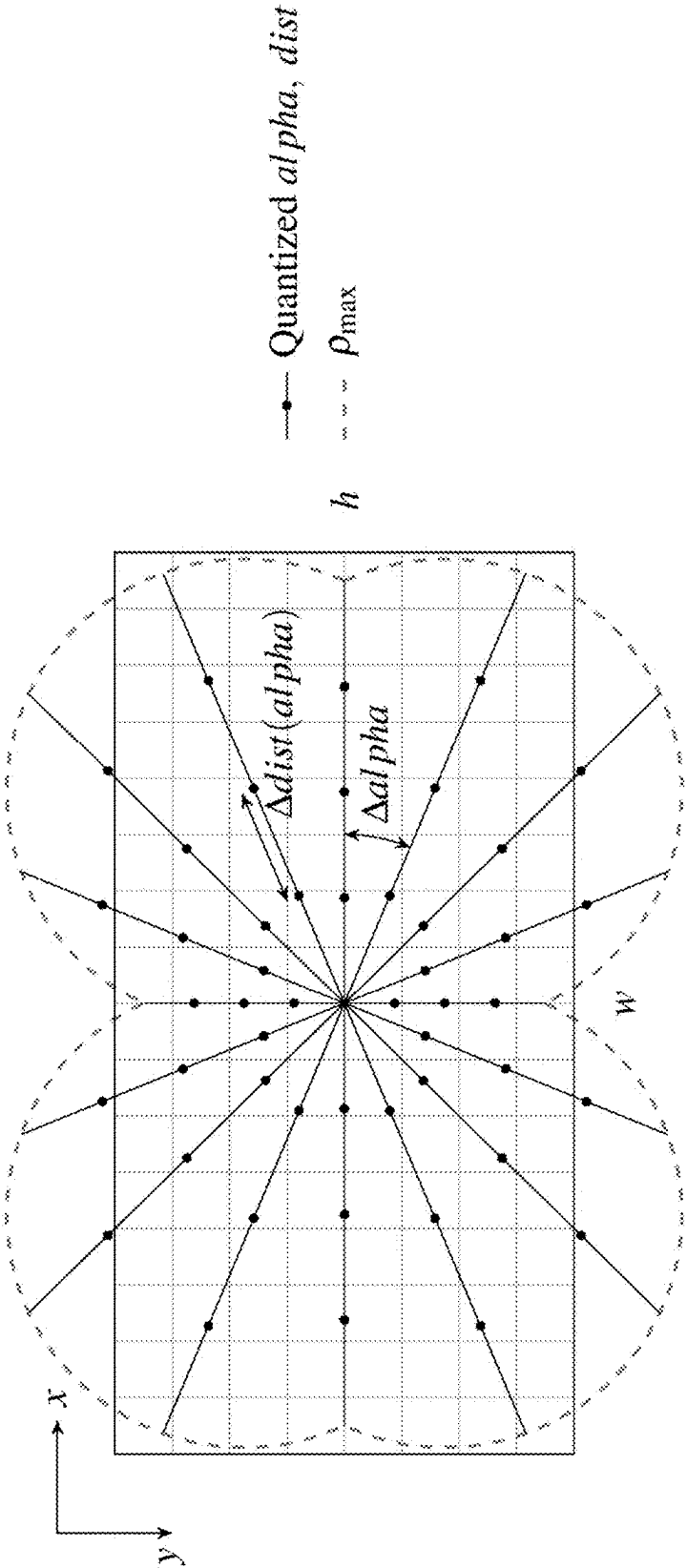


FIG. 16

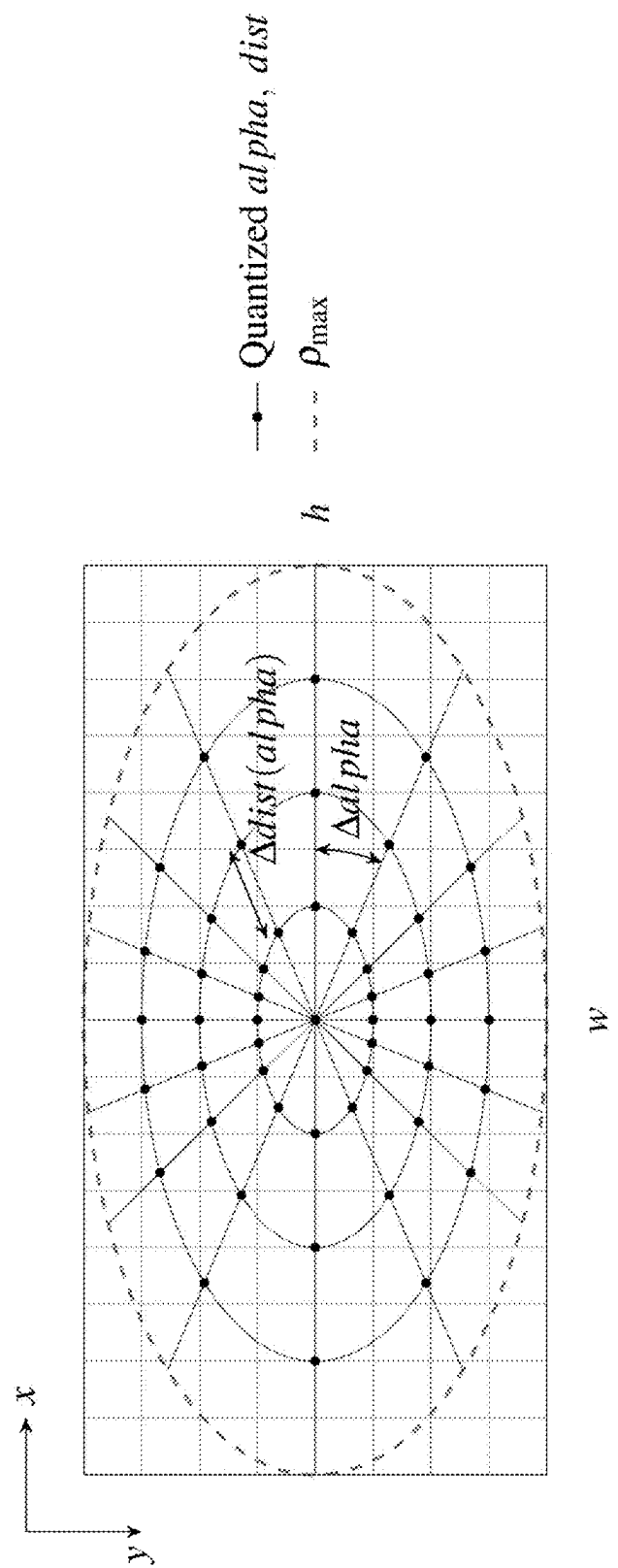


FIG. 17

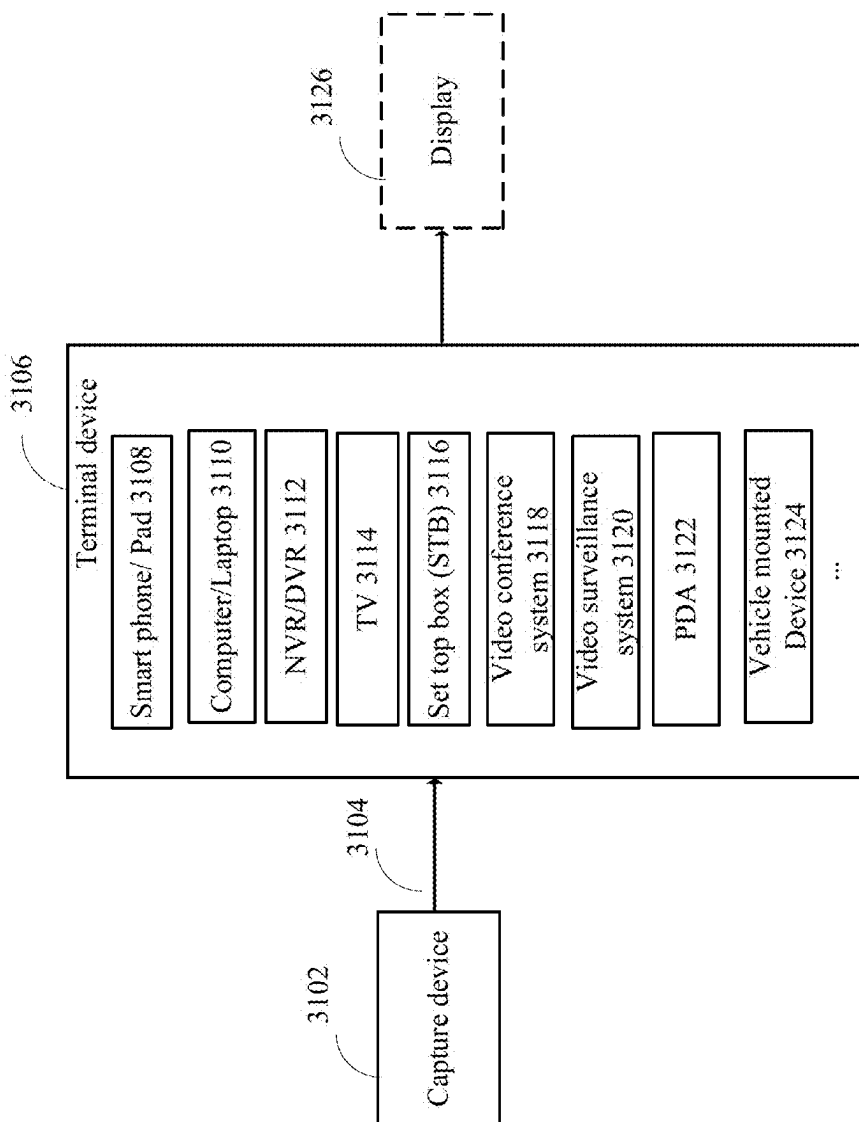


FIG. 18

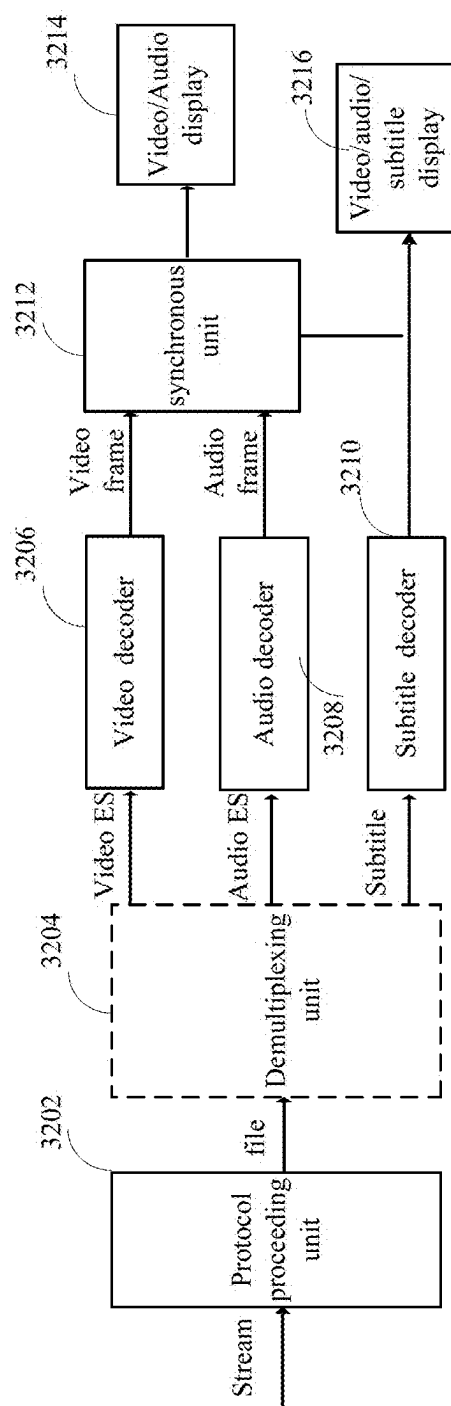


FIG. 19

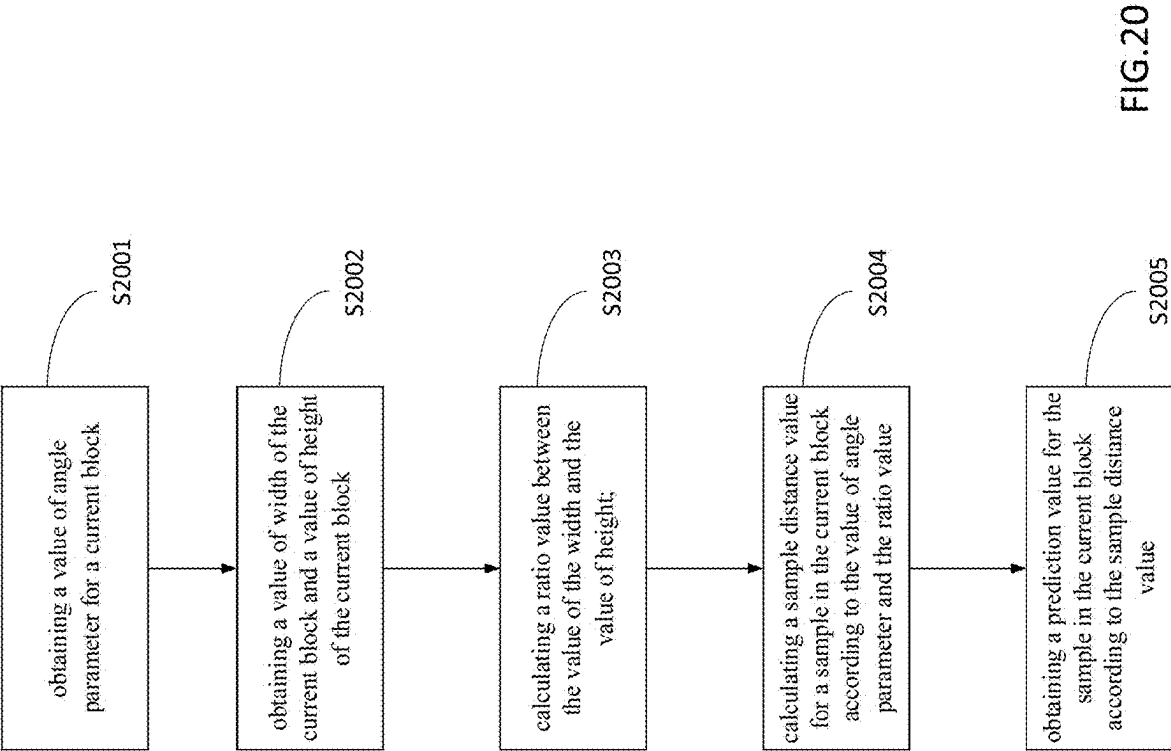


FIG.20

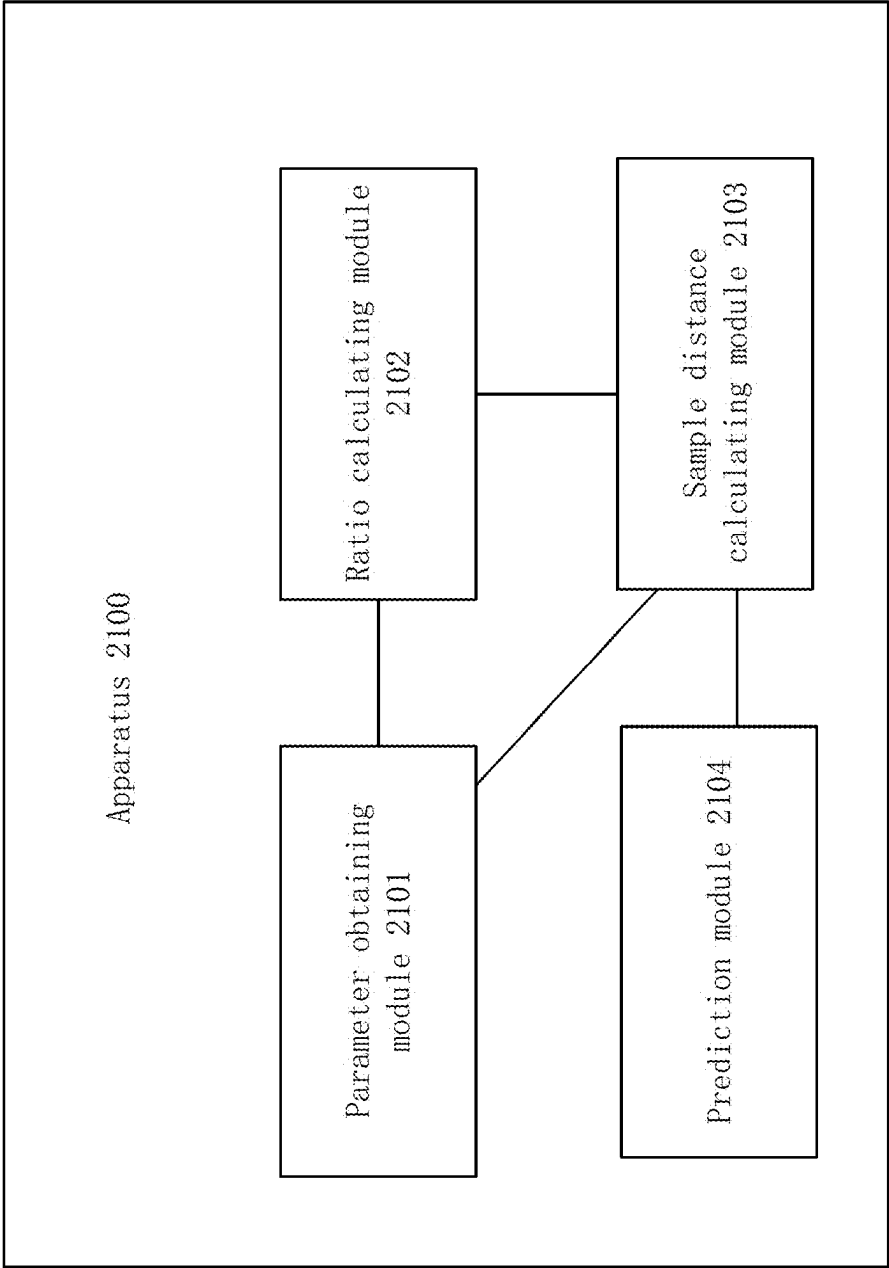


FIG.21

SAMPLE DISTANCE CALCULATION FOR GEOMETRIC PARTITION MODE

CROSS-REFERENCE TO RELATED APPLICATION

[0001] This application is a continuation of U.S. patent application Ser. No. 17/369,862, filed on Jul. 7, 2021, which is a continuation of International Application No. PCT/CN2020/097628, filed on Jun. 23, 2020, which claims priority to International Application No. PCT/EP2019/066685, filed on Jun. 24, 2019. All of the afore-mentioned patent applications are hereby incorporated by reference in their entireties.

TECHNICAL FIELD

[0002] Embodiments of the present disclosure generally relate to the field of picture processing, particularly to prediction for partitioning, and more particularly to sample distance calculation for geometric partition modes.

BACKGROUND

[0003] Video coding (video encoding and decoding) is used in a wide range of digital video applications, for example broadcast digital TV, video transmission over internet and mobile networks, real-time conversational applications such as video chat, video conferencing, DVD and Blu-ray discs, video content acquisition and editing systems, and camcorders of security applications.

[0004] The amount of video data needed to depict even a relatively short video can be substantial, which may result in difficulties when the data is to be streamed or otherwise communicated across a communications network with limited bandwidth capacity. Thus, video data is generally compressed before being communicated across modern day telecommunications networks. The size of a video could also be an issue when the video is stored on a storage device because memory resources may be limited. Video compression devices often use software and/or hardware at the source to code the video data prior to transmission or storage, thereby decreasing the quantity of data needed to represent digital video images. The compressed data is then received at the destination by a video decompression device that decodes the video data. With limited network resources and ever increasing demands of higher video quality, improved compression and decompression techniques that improve compression ratio with little to no sacrifice in picture quality are desirable.

SUMMARY

[0005] Embodiments of the present disclosure provide apparatuses and methods for encoding and decoding according to the independent claims.

[0006] The foregoing and other objects are achieved by the subject matter of the independent claims. Further implementation forms are apparent from the dependent claims, the description and the figures.

[0007] An embodiment of the disclosure provides a method of coding implemented by a decoding device or an encoding device, the method comprising: obtaining a value of an angle parameter for a current block; obtaining a value of a width of the current block and a value of a height of the current block; calculating a ratio value between the value of the width and the value of height; calculating a sample

distance value for a sample in the current block according to the value of angle parameter and the ratio value; and obtaining a prediction value for the sample in the current block according to the sample distance value.

[0008] In the embodiments of the disclosure, angular parameter and angle parameter are synonyms.

[0009] According to embodiments of the present disclosure, a ratio value between a value of a width of a block and a value of a height of the block is calculated. This ratio value is used to calculate a sample distance value and the prediction value is calculated much more accurately. Hence, the prediction residual may be reduced and the coding efficiency can be improved.

[0010] In an embodiment, the calculating a sample distance value for a sample in the current block according to the value of angle parameter and the ratio value comprises:

[0011] obtaining a first value according to the value of the angle parameter and the ratio value;

[0012] calculating the sample distance value for the sample in the current block according to the first value.

[0013] In an embodiment, the calculating the sample distance value for the sample in the current block according to the first value, comprises:

[0014] calculating the sample distance value for the sample in the current block according to the first value and the value of the angle parameter.

[0015] In an embodiment, the calculating the sample distance value for the sample in the current block according to the first value, comprises:

[0016] calculating the sample distance value for the sample in the current block according to the first value, the value of the angle parameter, a value of a distance index (e.g. the distance index of the current block), the value of the width and the value of height.

[0017] In an embodiment, the sample distance value represents the horizontal distance or the vertical distance, or a combination of the vertical and horizontal distance, of the sample to a separation line, wherein the separation line is used to divide a coding block into two sub-blocks.

[0018] In an embodiment, the obtaining a prediction value for the sample in the current block according to the sample distance value comprises:

[0019] calculating two weighting factors according to the sample distance value;

[0020] obtaining the prediction value for the sample in the current block according to a first prediction value, a second prediction value and the two weighting factors, e.g. “applying the two weighting factors to the 1st pred. value and the 2nd pred. value” or “applying the first weighting factor to the first prediction value and the second weighting factor to the second prediction value”.

[0021] In an embodiment, the ratio value is obtained according to,

$$\text{whRatio} = (W > H) ? W/H : H/W,$$

wherein whRatio is the ratio value, W is the value of the width, H is the value of the height.

[0022] In an embodiment, the value of angle parameter is an integer value, a value range for the value of angle parameter is between 0 to 31, including 0 and 31.

[0023] In an embodiment, the ratio value is obtained according to,

$$\text{whRatio} = (wIdx > hIdx) ? wIdx - hIdx : hIdx - wIdx,$$

[0024] where whRatio is the ratio value, W is the value of the width, H is the value of the height, a value of wIdx is equal to $\log 2(W)-3$, and a value of hIdx is equal to $\log 2(H)-3$.

[0025] In one implementation, the sample distance value is calculated according to,

$$\text{sample_dist} = ((2 * x) - W + K) * f1(\alpha) + ((2 * y) - H + K) * f2(\alpha) \text{ distanceIdx} * \text{stepD} * \text{scaleStep},$$

wherein sample_dist is the sample distance value, f1() and f2() are functions, and alpha is the value of the angle parameter; stepD is the first value, a value of scaleStep is equal to either the value of the width or the value of the height; x and y are the coordinates of the sample with respect to the top-left sample of the current block; and K is an integer offset value.

[0026] In an embodiment, the sample distance value is calculated according to,

$$\text{sample_dist} = ((2 * x) - W + K) * f(\text{dispIdxN}) + ((2 * y) - H + K) * f(\text{dispIdxS}) - \text{distanceIdx} * \text{stepD} * \text{scaleStep},$$

where sample_dist is the sample distance value, f() is a function, dispIdxN or dispIdxS are index values, stepD is the first value, a value of scaleStep is equal to either the value of the width or the value of the height, x and y are the coordinates of the sample with respect to the top-left sample of the current block, K is an integer offset value.

[0027] An embodiment of the present disclosure provides a video coding or decoding apparatus, wherein the apparatus comprises: parameter obtaining module, which is configured to obtain a value of an angle parameter for a current block; the parameter obtaining module is also configured to obtain a value of a width of the current block and a value of a height of the current block; ratio calculating module, which is configured to calculate a ratio value between the value of the width and the value of height; sample distance calculating module, which is configured to calculate a sample distance value for a sample in the current block according to the value of angle parameter and the ratio value; prediction module, which is configured to obtain a prediction value for the sample in the current block according to the sample distance value.

[0028] The method according to an embodiment of the disclosure can be performed by the apparatus according to the second aspect of the disclosure. Further features and implementation forms of the above methods correspond to the features and implementation forms of the apparatus according to the second aspect of the disclosure.

[0029] In an embodiment, a decoder (30) or an encoder (20) comprising processing circuitry for carrying out the method according to any one of the above embodiments and implementations is disclosed.

[0030] In an embodiment, a computer program product comprising a program code for performing the method according to any one of the above embodiments and implementations is disclosed.

[0031] In an embodiment, a decoder or an encoder is provided which comprises:

[0032] one or more processors; and

[0033] a non-transitory computer-readable storage medium coupled to the processors and storing programming for execution by the processors, wherein the programming, when executed by the processors, configures the decoder or the encoder to carry out the

method according to any one of the above embodiments and implementation is disclosed

[0034] In an embodiment, a non-transitory storage medium is provided, which includes an encoded bitstream decoded by an image decoding device, the bit stream being generated by dividing a frame of a video signal or an image signal into a plurality blocks, and including a plurality of syntax elements, wherein the plurality of syntax elements comprises an indicator (syntax) according to any one of the above embodiments and implementation is disclosed.

[0035] Details of one or more embodiments are set forth in the accompanying drawings and the description below. Other features, objects, and advantages will be apparent from the description, drawings, and claims.

BRIEF DESCRIPTION OF THE DRAWINGS

[0036] In the following embodiments of the disclosure are described in more detail with reference to the attached figures and drawings, in which:

[0037] FIG. 1A is a block diagram showing an example of a video coding system configured to implement embodiments of the disclosure;

[0038] FIG. 1B is a block diagram showing another example of a video coding system configured to implement embodiments of the disclosure;

[0039] FIG. 2 is a block diagram showing an example of a video encoder configured to implement embodiments of the disclosure;

[0040] FIG. 3 is a block diagram showing an example structure of a video decoder configured to implement embodiments of the disclosure;

[0041] FIG. 4 is a block diagram illustrating an example of an encoding apparatus or a decoding apparatus;

[0042] FIG. 5 is a block diagram illustrating another example of an encoding apparatus or a decoding apparatus;

[0043] FIG. 6a illustrated an example of Co-located block;

[0044] FIG. 6b illustrated an example of spatial neighbor blocks;

[0045] FIG. 7 illustrated some examples of triangular prediction mode;

[0046] FIG. 8 illustrated some examples of Sub-block prediction mode;

[0047] FIG. 9-12 show some examples about partition of a block;

[0048] FIG. 13 illustrated an example implementation of predefined lookup table for stepD;

[0049] FIG. 14 illustrated an example implementation of predefined lookup table for f();

[0050] FIG. 15 illustrated an example of the quantization aspect relating to the predefined lookup table for stepD;

[0051] FIG. 16 illustrated an example of a quantization scheme, where a maximum distance pmax is defined for a given coding block;

[0052] FIG. 17 illustrated an example of a quantization scheme, where an alternative maximum distance pmax is defined for a given coding block.

[0053] FIG. 18 is a block diagram showing an example structure of a content supply system which realizes a content delivery service.

[0054] FIG. 19 is a block diagram showing a structure of an example of a terminal device.

[0055] FIG. 20 is a flowchart showing an embodiment of a method embodiment according to the present disclosure.

[0056] FIG. 21 is a block diagram showing an embodiment of an apparatus embodiment according to the present disclosure.

[0057] In the following identical reference signs refer to identical or at least functionally equivalent features if not explicitly specified otherwise.

DETAILED DESCRIPTION OF THE EMBODIMENTS

[0058] In the following description, reference is made to the accompanying figures, which form part of the disclosure, and which show, by way of illustration, specific aspects of embodiments of the disclosure or specific aspects in which embodiments of the present disclosure may be used. It is understood that embodiments of the disclosure may be used in other aspects and comprise structural or logical changes not depicted in the figures. The following detailed description, therefore, is not to be taken in a limiting sense, and the scope of the present disclosure is defined by the appended claims.

[0059] For instance, it is understood that a disclosure in connection with a described method may also hold true for a corresponding device or system configured to perform the method and vice versa. For example, if one or a plurality of specific method operations are described, a corresponding device may include one or a plurality of units, e.g. functional units, to perform the described one or plurality of method operations (e.g. one unit performing the one or plurality of operations, or a plurality of units each performing one or more of the plurality of operations), even if such one or more units are not explicitly described or illustrated in the figures. On the other hand, for example, if a specific apparatus is described based on one or a plurality of units, e.g. functional units, a corresponding method may include one operation to perform the functionality of the one or plurality of units (e.g. one operation performing the functionality of the one or plurality of units, or a plurality of operations each performing the functionality of one or more of the plurality of units), even if such one or plurality of operations are not explicitly described or illustrated in the figures. Further, it is understood that the features of the various embodiments and/or aspects described herein may be combined with each other, unless specifically noted otherwise.

[0060] Video coding typically refers to the processing of a sequence of pictures, which form the video or video sequence. Instead of the term “picture” the term “frame” or “image” may be used as synonyms in the field of video coding. Video coding (or coding in general) comprises two parts video encoding and video decoding. Video encoding is performed at the source side, typically comprising processing (e.g. by compression) the original video pictures to reduce the amount of data required for representing the video pictures (for more efficient storage and/or transmission). Video decoding is performed at the destination side and typically comprises the inverse processing compared to the encoder to reconstruct the video pictures. Embodiments referring to “coding” of video pictures (or pictures in general) shall be understood to relate to “encoding” or “decoding” of video pictures or respective video sequences. The combination of the encoding part and the decoding part is also referred to as CODEC (Coding and Decoding).

[0061] In case of lossless video coding, the original video pictures can be reconstructed, i.e. the reconstructed video pictures have the same quality as the original video pictures

(assuming no transmission loss or other data loss during storage or transmission). In case of lossy video coding, further compression, e.g. by quantization, is performed, to reduce the amount of data representing the video pictures, which cannot be completely reconstructed at the decoder, i.e. the quality of the reconstructed video pictures is lower or worse compared to the quality of the original video pictures.

[0062] Several video coding standards belong to the group of “lossy hybrid video codecs” (i.e. combine spatial and temporal prediction in the sample domain and 2D transform coding for applying quantization in the transform domain). Each picture of a video sequence is typically partitioned into a set of non-overlapping blocks and the coding is typically performed on a block level. In other words, at the encoder the video is typically processed, i.e. encoded, on a block (video block) level, e.g. by using spatial (intra picture) prediction and/or temporal (inter picture) prediction to generate a prediction block, subtracting the prediction block from the current block (block currently processed/to be processed) to obtain a residual block, transforming the residual block and quantizing the residual block in the transform domain to reduce the amount of data to be transmitted (compression), whereas at the decoder the inverse processing compared to the encoder is applied to the encoded or compressed block to reconstruct the current block for representation. Furthermore, the encoder duplicates the decoder processing loop such that both will generate identical predictions (e.g. intra- and inter predictions) and/or re-constructions for processing, i.e. coding, the subsequent blocks.

[0063] In the following embodiments of a video coding system 10, a video encoder 20 and a video decoder 30 are described based on FIGS. 1 to 3.

[0064] FIG. 1A is a schematic block diagram illustrating an example coding system 10, e.g. a video coding system 10 (or short coding system 10) that may utilize techniques of this present disclosure. Video encoder 20 (or short encoder 20) and video decoder 30 (or short decoder 30) of video coding system 10 represent examples of devices that may be configured to perform techniques in accordance with various examples described in the present disclosure.

[0065] As shown in FIG. 1A, the coding system 10 comprises a source device 12 configured to provide encoded picture data 21 e.g. to a destination device 14 for decoding the encoded picture data 13.

[0066] The source device 12 comprises an encoder 20, and may additionally, i.e. optionally, comprise a picture source 16, a pre-processor (or pre-processing unit) 18, e.g. a picture pre-processor 18, and a communication interface or communication unit 22.

[0067] The picture source 16 may comprise or be any kind of picture capturing device, for example a camera for capturing a real-world picture, and/or any kind of a picture generating device, for example a computer-graphics processor for generating a computer animated picture, or any kind of other device for obtaining and/or providing a real-world picture, a computer generated picture (e.g. a screen content, a virtual reality (VR) picture) and/or any combination thereof (e.g. an augmented reality (AR) picture). The picture source may be any kind of memory or storage storing any of the aforementioned pictures.

[0068] In distinction to the pre-processor 18 and the processing performed by the pre-processing unit 18, the

picture or picture data 17 may also be referred to as raw picture or raw picture data 17.

[0069] Pre-processor 18 is configured to receive the (raw) picture data 17 and to perform pre-processing on the picture data 17 to obtain a pre-processed picture 19 or pre-processed picture data 19. Pre-processing performed by the pre-processor 18 may, e.g., comprise trimming, color format conversion (e.g. from RGB to YCbCr), color correction, or de-noising. It can be understood that the pre-processing unit 18 may be optional component.

[0070] The video encoder 20 is configured to receive the pre-processed picture data 19 and provide encoded picture data 21 (further details will be described below, e.g., based on FIG. 2).

[0071] Communication interface 22 of the source device 12 may be configured to receive the encoded picture data 21 and to transmit the encoded picture data 21 (or any further processed version thereof) over communication channel 13 to another device, e.g. the destination device 14 or any other device, for storage or direct reconstruction.

[0072] The destination device 14 comprises a decoder 30 (e.g. a video decoder 30), and may additionally, i.e. optionally, comprise a communication interface or communication unit 28, a post-processor 32 (or post-processing unit 32) and a display device 34.

[0073] The communication interface 28 of the destination device 14 is configured receive the encoded picture data 21 (or any further processed version thereof), e.g. directly from the source device 12 or from any other source, e.g. a storage device, e.g. an encoded picture data storage device, and provide the encoded picture data 21 to the decoder 30.

[0074] The communication interface 22 and the communication interface 28 may be configured to transmit or receive the encoded picture data 21 or encoded data 13 via a direct communication link between the source device 12 and the destination device 14, e.g. a direct wired or wireless connection, or via any kind of network, e.g. a wired or wireless network or any combination thereof, or any kind of private and public network, or any kind of combination thereof.

[0075] The communication interface 22 may be, e.g., configured to package the encoded picture data 21 into an appropriate format, e.g. packets, and/or process the encoded picture data using any kind of transmission encoding or processing for transmission over a communication link or communication network.

[0076] The communication interface 28, forming the counterpart of the communication interface 22, may be, e.g., configured to receive the transmitted data and process the transmission data using any kind of corresponding transmission decoding or processing and/or de-packaging to obtain the encoded picture data 21.

[0077] Both, communication interface 22 and communication interface 28 may be configured as unidirectional communication interfaces as indicated by the arrow for the communication channel 13 in FIG. 1A pointing from the source device 12 to the destination device 14, or bi-directional communication interfaces, and may be configured, e.g. to send and receive messages, e.g. to set up a connection, to acknowledge and exchange any other information related to the communication link and/or data transmission, e.g. encoded picture data transmission.

[0078] The decoder 30 is configured to receive the encoded picture data 21 and provide decoded picture data 31

or a decoded picture 31 (further details will be described below, e.g., based on FIG. 3 or FIG. 5).

[0079] The post-processor 32 of destination device 14 is configured to post-process the decoded picture data 31 (also called reconstructed picture data), e.g. the decoded picture 31, to obtain post-processed picture data 33, e.g. a post-processed picture 33. The post-processing performed by the post-processing unit 32 may comprise, e.g. color format conversion (e.g. from YCbCr to RGB), color correction, trimming, or re-sampling, or any other processing, e.g. for preparing the decoded picture data 31 for display, e.g. by display device 34.

[0080] The display device 34 of the destination device 14 is configured to receive the post-processed picture data 33 for displaying the picture, e.g. to a user or viewer. The display device 34 may be or comprise any kind of display for representing the reconstructed picture, e.g. an integrated or external display or monitor. The displays may, e.g. comprise liquid crystal displays (LCD), organic light emitting diodes (OLED) displays, plasma displays, projectors, micro LED displays, liquid crystal on silicon (LCoS), digital light processor (DLP) or any kind of other display.

[0081] Although FIG. 1A depicts the source device 12 and the destination device 14 as separate devices, embodiments of devices may also comprise both or both functionalities, the source device 12 or corresponding functionality and the destination device 14 or corresponding functionality. In such embodiments the source device 12 or corresponding functionality and the destination device 14 or corresponding functionality may be implemented using the same hardware and/or software or by separate hardware and/or software or any combination thereof.

[0082] As will be apparent for the skilled person based on the description, the existence and (exact) split of functionalities of the different units or functionalities within the source device 12 and/or destination device 14 as shown in FIG. 1A may vary depending on the actual device and application.

[0083] The encoder 20 (e.g. a video encoder 20) or the decoder 30 (e.g. a video decoder 30) or both encoder 20 and decoder 30 may be implemented via processing circuitry as shown in FIG. 1B, such as one or more microprocessors, digital signal processors (DSPs), application-specific integrated circuits (ASICs), field-programmable gate arrays (FPGAs), discrete logic, hardware, video coding dedicated or any combinations thereof. The encoder 20 may be implemented via processing circuitry 46 to embody the various modules as discussed with respect to encoder 20 of FIG. 2 and/or any other encoder system or subsystem described herein. The decoder 30 may be implemented via processing circuitry 46 to embody the various modules as discussed with respect to decoder 30 of FIG. 3 and/or any other decoder system or subsystem described herein. The processing circuitry may be configured to perform the various operations as discussed later. As shown in FIG. 5, if the techniques are implemented partially in software, a device may store instructions for the software in a suitable, non-transitory computer-readable storage medium and may execute the instructions in hardware using one or more processors to perform the techniques of this disclosure. Either of video encoder 20 and video decoder 30 may be integrated as part of a combined encoder/decoder (CODEC) in a single device, for example, as shown in FIG. 1B.

[0084] Source device 12 and destination device 14 may comprise any of a wide range of devices, including any kind of handheld or stationary devices, e.g. notebook or laptop computers, mobile phones, smart phones, tablets or tablet computers, cameras, desktop computers, set-top boxes, televisions, display devices, digital media players, video gaming consoles, video streaming devices (such as content services servers or content delivery servers), broadcast receiver device, broadcast transmitter device, or the like and may use no or any kind of operating system. In some cases, the source device 12 and the destination device 14 may be equipped for wireless communication. Thus, the source device 12 and the destination device 14 may be wireless communication devices.

[0085] In some cases, video coding system 10 illustrated in FIG. 1A is merely an example and the techniques of the present disclosure may apply to video coding settings (e.g., video encoding or video decoding) that do not necessarily include any data communication between the encoding and decoding devices. In other examples, data is retrieved from a local memory, streamed over a network, or the like. A video encoding device may encode and store data to memory, and/or a video decoding device may retrieve and decode data from memory. In some examples, the encoding and decoding is performed by devices that do not communicate with one another, but simply encode data to memory and/or retrieve and decode data from memory.

[0086] For convenience of description, embodiments of the disclosure are described herein, for example, by reference to High-Efficiency Video Coding (HEVC) or to the reference software of Versatile Video coding (VVC), the next generation video coding standard developed by the Joint Collaboration Team on Video Coding (JCT-VC) of ITU-T Video Coding Experts Group (VCEG) and ISO/IEC Motion Picture Experts Group (MPEG). One of ordinary skill in the art will understand that embodiments of the disclosure are not limited to HEVC or VVC.

Encoder and Encoding Method

[0087] FIG. 2 shows a schematic block diagram of an example video encoder 20 that is configured to implement the techniques of the present disclosure. In the example of FIG. 2, the video encoder 20 comprises an input 201 (or input interface 201), a residual calculation unit 204, a transform processing unit 206, a quantization unit 208, an inverse quantization unit 210, and inverse transform processing unit 212, a reconstruction unit 214, a loop filter unit 220, a decoded picture buffer (DPB) 230, a mode selection unit 260, an entropy encoding unit 270 and an output 272 (or output interface 272). The mode selection unit 260 may include an inter prediction unit 244, an intra prediction unit 254 and a partitioning unit 262. Inter prediction unit 244 may include a motion estimation unit and a motion compensation unit (not shown). A video encoder 20 as shown in FIG. 2 may also be referred to as hybrid video encoder or a video encoder according to a hybrid video codec.

[0088] The residual calculation unit 204, the transform processing unit 206, the quantization unit 208, the mode selection unit 260 may be referred to as forming a forward signal path of the encoder 20, whereas the inverse quantization unit 210, the inverse transform processing unit 212, the reconstruction unit 214, the buffer 216, the loop filter 220, the decoded picture buffer (DPB) 230, the inter prediction unit 244 and the intra-prediction unit 254 may be

referred to as forming a backward signal path of the video encoder 20, wherein the backward signal path of the video encoder 20 corresponds to the signal path of the decoder (see video decoder 30 in FIG. 3). The inverse quantization unit 210, the inverse transform processing unit 212, the reconstruction unit 214, the loop filter 220, the decoded picture buffer (DPB) 230, the inter prediction unit 244 and the intra-prediction unit 254 are also referred to forming the “built-in decoder” of video encoder 20.

Pictures & Picture Partitioning (Pictures & Blocks)

[0089] The encoder 20 may be configured to receive, e.g. via input 201, a picture 17 (or picture data 17), e.g. picture of a sequence of pictures forming a video or video sequence. The received picture or picture data may also be a pre-processed picture 19 (or pre-processed picture data 19). For sake of simplicity the following description refers to the picture 17. The picture 17 may also be referred to as current picture or picture to be coded (in particular in video coding to distinguish the current picture from other pictures, e.g. previously encoded and/or decoded pictures of the same video sequence, i.e. the video sequence which also comprises the current picture).

[0090] A (digital) picture is or can be regarded as a two-dimensional array or matrix of samples with intensity values. A sample in the array may also be referred to as pixel (short form of picture element) or a pel. The number of samples in horizontal and vertical direction (or axis) of the array or picture define the size and/or resolution of the picture. For representation of color, typically three color components are employed, i.e. the picture may be represented or include three sample arrays. In RGB format or color space a picture comprises a corresponding red, green and blue sample array. However, in video coding each pixel is typically represented in a luminance and chrominance format or color space, e.g. YCbCr, which comprises a luminance component indicated by Y (sometimes also L is used instead) and two chrominance components indicated by Cb and Cr. The luminance (or short luma) component Y represents the brightness or grey level intensity (e.g. like in a grey-scale picture), while the two chrominance (or short chroma) components Cb and Cr represent the chromaticity or color information components. Accordingly, a picture in YCbCr format comprises a luminance sample array of luminance sample values (Y), and two chrominance sample arrays of chrominance values (Cb and Cr). Pictures in RGB format may be converted or transformed into YCbCr format and vice versa, the process is also known as color transformation or conversion. If a picture is monochrome, the picture may comprise only a luminance sample array. Accordingly, a picture may be, for example, an array of luma samples in monochrome format or an array of luma samples and two corresponding arrays of chroma samples in 4:2:0, 4:2:2, and 4:4:4 colour format.

[0091] Embodiments of the video encoder 20 may comprise a picture partitioning unit (not depicted in FIG. 2) configured to partition the picture 17 into a plurality of (typically non-overlapping) picture blocks 203. These blocks may also be referred to as root blocks, macro blocks (H.264/AVC) or coding tree blocks (CTB) or coding tree units (CTU) (H.265/HEVC and VVC). The picture partitioning unit may be configured to use the same block size for all pictures of a video sequence and the corresponding grid defining the block size, or to change the block size between

pictures or subsets or groups of pictures, and partition each picture into the corresponding blocks.

[0092] In further embodiments, the video encoder may be configured to receive directly a block **203** of the picture **17**, e.g. one, several or all blocks forming the picture **17**. The picture block **203** may also be referred to as current picture block or picture block to be coded.

[0093] Like the picture **17**, the picture block **203** again is or can be regarded as a two-dimensional array or matrix of samples with intensity values (sample values), although of smaller dimension than the picture **17**. In other words, the block **203** may comprise, e.g., one sample array (e.g. a luma array in case of a monochrome picture **17**, or a luma or chroma array in case of a color picture) or three sample arrays (e.g. a luma and two chroma arrays in case of a color picture **17**) or any other number and/or kind of arrays depending on the color format applied. The number of samples in horizontal and vertical direction (or axis) of the block **203** define the size of block **203**. Accordingly, a block may, for example, an MxN (M-column by N-row) array of samples, or an MxN array of transform coefficients.

[0094] Embodiments of the video encoder **20** as shown in FIG. **2** may be configured encode the picture **17** block by block, e.g. the encoding and prediction is performed per block **203**.

Residual Calculation

[0095] The residual calculation unit **204** may be configured to calculate a residual block **205** (also referred to as residual **205**) based on the picture block **203** and a prediction block **265** (further details about the prediction block **265** are provided later), e.g. by subtracting sample values of the prediction block **265** from sample values of the picture block **203**, sample by sample (pixel by pixel) to obtain the residual block **205** in the sample domain.

Transform

[0096] The transform processing unit **206** may be configured to apply a transform, e.g. a discrete cosine transform (DCT) or discrete sine transform (DST), on the sample values of the residual block **205** to obtain transform coefficients **207** in a transform domain. The transform coefficients **207** may also be referred to as transform residual coefficients and represent the residual block **205** in the transform domain.

[0097] The transform processing unit **206** may be configured to apply integer approximations of DCT/DST, such as the transforms specified for H.265/HEVC. Compared to an orthogonal DCT transform, such integer approximations are typically scaled by a certain factor. In order to preserve the norm of the residual block which is processed by forward and inverse transforms, additional scaling factors are applied as part of the transform process. The scaling factors are typically chosen based on certain constraints like scaling factors being a power of two for shift operations, bit depth of the transform coefficients, tradeoff between accuracy and implementation costs, etc. Specific scaling factors are, for example, specified for the inverse transform, e.g. by inverse transform processing unit **212** (and the corresponding inverse transform, e.g. by inverse transform processing unit **312** at video decoder **30**) and corresponding scaling factors for the forward transform, e.g. by transform processing unit **206**, at an encoder **20** may be specified accordingly.

[0098] Embodiments of the video encoder **20** (respectively transform processing unit **206**) may be configured to output transform parameters, e.g. a type of transform or transforms, e.g. directly or encoded or compressed via the entropy encoding unit **270**, so that, e.g., the video decoder **30** may receive and use the transform parameters for decoding.

Quantization

[0099] The quantization unit **208** may be configured to quantize the transform coefficients **207** to obtain quantized coefficients **209**, e.g. by applying scalar quantization or vector quantization. The quantized coefficients **209** may also be referred to as quantized transform coefficients **209** or quantized residual coefficients **209**.

[0100] The quantization process may reduce the bit depth associated with some or all of the transform coefficients **207**. For example, an n-bit transform coefficient may be rounded down to an m-bit Transform coefficient during quantization, where n is greater than m. The degree of quantization may be modified by adjusting a quantization parameter (QP). For example for scalar quantization, different scaling may be applied to achieve finer or coarser quantization. Smaller quantization step sizes correspond to finer quantization, whereas larger quantization step sizes correspond to coarser quantization. The applicable quantization step size may be indicated by a quantization parameter (QP). The quantization parameter may for example be an index to a predefined set of applicable quantization step sizes. For example, small quantization parameters may correspond to fine quantization (small quantization step sizes) and large quantization parameters may correspond to coarse quantization (large quantization step sizes) or vice versa. The quantization may include division by a quantization step size and a corresponding and/or the inverse dequantization, e.g. by inverse quantization unit **210**, may include multiplication by the quantization step size. Embodiments according to some standards, e.g. HEVC, may be configured to use a quantization parameter to determine the quantization step size. Generally, the quantization step size may be calculated based on a quantization parameter using a fixed point approximation of an equation including division. Additional scaling factors may be introduced for quantization and dequantization to restore the norm of the residual block, which might get modified because of the scaling used in the fixed point approximation of the equation for quantization step size and quantization parameter. In one example implementation, the scaling of the inverse transform and dequantization might be combined. Alternatively, customized quantization tables may be used and signaled from an encoder to a decoder, e.g. in a bitstream. The quantization is a lossy operation, wherein the loss increases with increasing quantization step sizes.

[0101] Embodiments of the video encoder **20** (respectively quantization unit **208**) may be configured to output quantization parameters (QP), e.g. directly or encoded via the entropy encoding unit **270**, so that, e.g., the video decoder **30** may receive and apply the quantization parameters for decoding.

Inverse Quantization

[0102] The inverse quantization unit **210** is configured to apply the inverse quantization of the quantization unit **208** on the quantized coefficients to obtain dequantized coefficients.

cients **211**, e.g. by applying the inverse of the quantization scheme applied by the quantization unit **208** based on or using the same quantization step size as the quantization unit **208**. The dequantized coefficients **211** may also be referred to as dequantized residual coefficients **211** and correspond—although typically not identical to the transform coefficients due to the loss by quantization—to the transform coefficients **207**.

Inverse Transform

[0103] The inverse transform processing unit **212** is configured to apply the inverse transform of the transform applied by the transform processing unit **206**, e.g. an inverse discrete cosine transform (DCT) or inverse discrete sine transform (DST) or other inverse transforms, to obtain a reconstructed residual block **213** (or corresponding dequantized coefficients **213**) in the sample domain. The reconstructed residual block **213** may also be referred to as transform block **213**.

Reconstruction

[0104] The reconstruction unit **214** (e.g. adder or summer **214**) is configured to add the transform block **213** (i.e. reconstructed residual block **213**) to the prediction block **265** to obtain a reconstructed block **215** in the sample domain, e.g. by adding—sample by sample—the sample values of the reconstructed residual block **213** and the sample values of the prediction block **265**.

Filtering

[0105] The loop filter unit **220** (or short “loop filter” **220**), is configured to filter the reconstructed block **215** to obtain a filtered block **221**, or in general, to filter reconstructed samples to obtain filtered samples. The loop filter unit is, e.g., configured to smooth pixel transitions, or otherwise improve the video quality. The loop filter unit **220** may comprise one or more loop filters such as a de-blocking filter, a sample-adaptive offset (SAO) filter or one or more other filters, e.g. a bilateral filter, an adaptive loop filter (ALF), a sharpening, a smoothing filters or a collaborative filters, or any combination thereof. Although the loop filter unit **220** is shown in FIG. 2 as being an in loop filter, in other configurations, the loop filter unit **220** may be implemented as a post loop filter. The filtered block **221** may also be referred to as filtered reconstructed block **221**.

[0106] Embodiments of the video encoder **20** (respectively loop filter unit **220**) may be configured to output loop filter parameters (such as sample adaptive offset information), e.g. directly or encoded via the entropy encoding unit **270**, so that, e.g., a decoder **30** may receive and apply the same loop filter parameters or respective loop filters for decoding.

Decoded Picture Buffer

[0107] The decoded picture buffer (DPB) **230** may be a memory that stores reference pictures, or in general reference picture data, for encoding video data by video encoder **20**. The DPB **230** may be formed by any of a variety of memory devices, such as dynamic random access memory (DRAM), including synchronous DRAM (SDRAM), magnetoresistive RAM (MRAM), resistive RAM (RRAM), or other types of memory devices. The decoded picture buffer (DPB) **230** may be configured to store one or more filtered

blocks **221**. The decoded picture buffer **230** may be further configured to store other previously filtered blocks, e.g. previously reconstructed and filtered blocks **221**, of the same current picture or of different pictures, e.g. previously reconstructed pictures, and may provide complete previously reconstructed, i.e. decoded, pictures (and corresponding reference blocks and samples) and/or a partially reconstructed current picture (and corresponding reference blocks and samples), for example for inter prediction. The decoded picture buffer (DPB) **230** may be also configured to store one or more unfiltered reconstructed blocks **215**, or in general unfiltered reconstructed samples, e.g. if the reconstructed block **215** is not filtered by loop filter unit **220**, or any other further processed version of the reconstructed blocks or samples.

Mode Selection (Partitioning & Prediction)

[0108] The mode selection unit **260** comprises partitioning unit **262**, inter-prediction unit **244** and intra-prediction unit **254**, and is configured to receive or obtain original picture data, e.g. an original block **203** (current block **203** of the current picture **17**), and reconstructed picture data, e.g. filtered and/or unfiltered reconstructed samples or blocks of the same (current) picture and/or from one or a plurality of previously decoded pictures, e.g. from decoded picture buffer **230** or other buffers (e.g. line buffer, not shown). The reconstructed picture data is used as reference picture data for prediction, e.g. inter-prediction or intra-prediction, to obtain a prediction block **265** or predictor **265**.

[0109] Mode selection unit **260** may be configured to determine or select a partitioning for a current block prediction mode (including no partitioning) and a prediction mode (e.g. an intra or inter prediction mode) and generate a corresponding prediction block **265**, which is used for the calculation of the residual block **205** and for the reconstruction of the reconstructed block **215**.

[0110] Embodiments of the mode selection unit **260** may be configured to select the partitioning and the prediction mode (e.g. from those supported by or available for mode selection unit **260**), which provide the best match or in other words the minimum residual (minimum residual means better compression for transmission or storage), or a minimum signaling overhead (minimum signaling overhead means better compression for transmission or storage), or which considers or balances both. The mode selection unit **260** may be configured to determine the partitioning and prediction mode based on rate distortion optimization (RDO), i.e. select the prediction mode which provides a minimum rate distortion. Terms like “best”, “minimum”, “optimum” etc. in this context do not necessarily refer to an overall “best”, “minimum”, “optimum”, etc. but may also refer to the fulfillment of a termination or selection criterion like a value exceeding or falling below a threshold or other constraints leading potentially to a “sub-optimum selection” but reducing complexity and processing time.

[0111] In other words, the partitioning unit **262** may be configured to partition the block **203** into smaller block partitions or sub-blocks (which form again blocks), e.g. iteratively using quad-tree-partitioning (QT), binary partitioning (BT) or triple-tree-partitioning (TT) or any combination thereof, and to perform, e.g., the prediction for each of the block partitions or sub-blocks, wherein the mode selection comprises the selection of the tree-structure of the

partitioned block **203** and the prediction modes are applied to each of the block partitions or sub-blocks.

[0112] In the following the partitioning (e.g. by partitioning unit **260**) and prediction processing (by inter-prediction unit **244** and intra-prediction unit **254**) performed by an example video encoder **20** will be explained in more detail.

Partitioning

[0113] The partitioning unit **262** may partition (or split) a current block **203** into smaller partitions, e.g. smaller blocks of square or rectangular size. These smaller blocks (which may also be referred to as sub-blocks) may be further partitioned into even smaller partitions. This is also referred to tree-partitioning or hierarchical tree-partitioning, wherein a root block, e.g. at root tree-level 0 (hierarchy-level 0, depth 0), may be recursively partitioned, e.g. partitioned into two or more blocks of a next lower tree-level, e.g. nodes at tree-level 1 (hierarchy-level 1, depth 1), wherein these blocks may be again partitioned into two or more blocks of a next lower level, e.g. tree-level 2 (hierarchy-level 2, depth 2), etc. until the partitioning is terminated, e.g. because a termination criterion is fulfilled, e.g. a maximum tree depth or minimum block size is reached. Blocks which are not further partitioned are also referred to as leaf-blocks or leaf nodes of the tree. A tree using partitioning into two partitions is referred to as binary-tree (BT), a tree using partitioning into three partitions is referred to as ternary-tree (TT), and a tree using partitioning into four partitions is referred to as quad-tree (QT).

[0114] As mentioned before, the term “block” as used herein may be a portion, in particular a square or rectangular portion, of a picture. With reference, for example, to HEVC and VVC, the block may be or correspond to a coding tree unit (CTU), a coding unit (CU), prediction unit (PU), and transform unit (TU) and/or to the corresponding blocks, e.g. a coding tree block (CTB), a coding block (CB), a transform block (TB) or prediction block (PB).

[0115] For example, a coding tree unit (CTU) may be or comprise a CTB of luma samples, two corresponding CTBs of chroma samples of a picture that has three sample arrays, or a CTB of samples of a monochrome picture or a picture that is coded using three separate colour planes and syntax structures used to code the samples. Correspondingly, a coding tree block (CTB) may be an $N \times N$ block of samples for some value of N such that the division of a component into CTBs is a partitioning. A coding unit (CU) may be or comprise a coding block of luma samples, two corresponding coding blocks of chroma samples of a picture that has three sample arrays, or a coding block of samples of a monochrome picture or a picture that is coded using three separate colour planes and syntax structures used to code the samples. Correspondingly a coding block (CB) may be an $M \times N$ block of samples for some values of M and N such that the division of a CTB into coding blocks is a partitioning.

[0116] In embodiments, e.g., according to HEVC, a coding tree unit (CTU) may be split into CUs by using a quad-tree structure denoted as coding tree. The decision whether to code a picture area using inter-picture (temporal) or intra-picture (spatial) prediction is made at the CU level. Each CU can be further split into one, two or four PUs according to the PU splitting type. Inside one PU, the same prediction process is applied and the relevant information is transmitted to the decoder on a PU basis. After obtaining the residual block by applying the prediction process based on

the PU splitting type, a CU can be partitioned into transform units (TUs) according to another quadtree structure similar to the coding tree for the CU.

[0117] In embodiments, e.g., according to the latest video coding standard currently in development, which is referred to as Versatile Video Coding (VVC), Quad-tree and binary tree (QTBT) partitioning is used to partition a coding block. In the QTBT block structure, a CU can have either a square or rectangular shape. For example, a coding tree unit (CTU) is first partitioned by a quadtree structure. The quadtree leaf nodes are further partitioned by a binary tree or ternary (or triple) tree structure. The partitioning tree leaf nodes are called coding units (CUs), and that segmentation is used for prediction and transform processing without any further partitioning. This means that the CU, PU and TU have the same block size in the QTBT coding block structure. In parallel, multiple partition, for example, triple tree partition was also proposed to be used together with the QTBT block structure.

[0118] In one example, the mode selection unit **260** of video encoder **20** may be configured to perform any combination of the partitioning techniques described herein.

[0119] As described above, the video encoder **20** is configured to determine or select the best or an optimum prediction mode from a set of (pre-determined) prediction modes. The set of prediction modes may comprise, e.g., intra-prediction modes and/or inter-prediction modes.

Intra-Prediction

[0120] The set of intra-prediction modes may comprise 35 different intra-prediction modes, e.g. non-directional modes like DC (or mean) mode and planar mode, or directional modes, e.g. as defined in HEVC, or may comprise 67 different intra-prediction modes, e.g. non-directional modes like DC (or mean) mode and planar mode, or directional modes, e.g. as defined for VVC.

[0121] The intra-prediction unit **254** is configured to use reconstructed samples of neighboring blocks of the same current picture to generate an intra-prediction block **265** according to an intra-prediction mode of the set of intra-prediction modes.

[0122] The intra prediction unit **254** (or in general the mode selection unit **260**) is further configured to output intra-prediction parameters (or in general information indicative of the selected intra prediction mode for the block) to the entropy encoding unit **270** in form of syntax elements **266** for inclusion into the encoded picture data **21**, so that, e.g., the video decoder **30** may receive and use the prediction parameters for decoding.

Inter-Prediction

[0123] The set of (or possible) inter-prediction modes depends on the available reference pictures (i.e. previous at least partially decoded pictures, e.g. stored in DBP **230**) and other inter-prediction parameters, e.g. whether the whole reference picture or only a part, e.g. a search window area around the area of the current block, of the reference picture is used for searching for a best matching reference block, and/or e.g. whether pixel interpolation is applied, e.g. half/semi-pel and/or quarter-pel interpolation, or not.

[0124] Additional to the above prediction modes, skip mode and/or direct mode may be applied.

[0125] The inter prediction unit **244** may include a motion estimation (ME) unit and a motion compensation (MC) unit (both not shown in FIG. 2). The motion estimation unit may be configured to receive or obtain the picture block **203** (current picture block **203** of the current picture **17**) and a decoded picture **231**, or at least one or a plurality of previously reconstructed blocks, e.g. reconstructed blocks of one or a plurality of other/different previously decoded pictures **231**, for motion estimation. E.g. a video sequence may comprise the current picture and the previously decoded pictures **231**, or in other words, the current picture and the previously decoded pictures **231** may be part of or form a sequence of pictures forming a video sequence.

[0126] The encoder **20** may, e.g., be configured to select a reference block from a plurality of reference blocks of the same or different pictures of the plurality of other pictures and provide a reference picture (or reference picture index) and/or an offset (spatial offset) between the position (x, y coordinates) of the reference block and the position of the current block as inter prediction parameters to the motion estimation unit. This offset is also called motion vector (MV).

[0127] The motion compensation unit is configured to obtain, e.g. receive, an inter prediction parameter and to perform inter prediction based on or using the inter prediction parameter to obtain an inter prediction block **265**. Motion compensation, performed by the motion compensation unit, may involve fetching or generating the prediction block based on the motion/block vector determined by motion estimation, possibly performing interpolations to sub-pixel precision. Interpolation filtering may generate additional pixel samples from known pixel samples, thus potentially increasing the number of candidate prediction blocks that may be used to code a picture block. Upon receiving the motion vector for the PU of the current picture block, the motion compensation unit may locate the prediction block to which the motion vector points in one of the reference picture lists.

[0128] Motion compensation unit may also generate syntax elements associated with the blocks and the video slice for use by video decoder **30** in decoding the picture blocks of the video slice.

Entropy Coding

[0129] The entropy encoding unit **270** is configured to apply, for example, an entropy encoding algorithm or scheme (e.g. a variable length coding (VLC) scheme, an context adaptive VLC scheme (CAVLC), an arithmetic coding scheme, a binarization, a context adaptive binary arithmetic coding (CABAC), syntax-based context-adaptive binary arithmetic coding (SBAC), probability interval partitioning entropy (PIPE) coding or another entropy encoding methodology or technique) or bypass (no compression) on the quantized coefficients **209**, inter prediction parameters, intra prediction parameters, loop filter parameters and/or other syntax elements to obtain encoded picture data **21** which can be output via the output **272**, e.g. in the form of an encoded bitstream **21**, so that, e.g., the video decoder **30** may receive and use the parameters for decoding. The encoded bitstream **21** may be transmitted to video decoder **30**, or stored in a memory for later transmission or retrieval by video decoder **30**.

[0130] Other structural variations of the video encoder **20** can be used to encode the video stream. For example, a

non-transform based encoder **20** can quantize the residual signal directly without the transform processing unit **206** for certain blocks or frames. In another implementation, an encoder **20** can have the quantization unit **208** and the inverse quantization unit **210** combined into a single unit.

Decoder and Decoding Method

[0131] FIG. 3 shows an example of a video decoder **30** that is configured to implement the techniques of this present disclosure. The video decoder **30** is configured to receive encoded picture data **21** (e.g. encoded bitstream **21**), e.g. encoded by encoder **20**, to obtain a decoded picture **331**. The encoded picture data or bitstream comprises information for decoding the encoded picture data, e.g. data that represents picture blocks of an encoded video slice and associated syntax elements.

[0132] In the example of FIG. 3, the decoder **30** comprises an entropy decoding unit **304**, an inverse quantization unit **310**, an inverse transform processing unit **312**, a reconstruction unit **314** (e.g. a summer **314**), a loop filter **320**, a decoded picture buffer (DPB) **330**, an inter prediction unit **344** and an intra prediction unit **354**. Inter prediction unit **344** may be or include a motion compensation unit. Video decoder **30** may, in some examples, perform a decoding pass generally reciprocal to the encoding pass described with respect to video encoder **100** from FIG. 2.

[0133] As explained with regard to the encoder **20**, the inverse quantization unit **210**, the inverse transform processing unit **212**, the reconstruction unit **214** the loop filter **220**, the decoded picture buffer (DPB) **230**, the inter prediction unit **344** and the intra prediction unit **354** are also referred to as forming the “built-in decoder” of video encoder **20**. Accordingly, the inverse quantization unit **310** may be identical in function to the inverse quantization unit **110**, the inverse transform processing unit **312** may be identical in function to the inverse transform processing unit **212**, the reconstruction unit **314** may be identical in function to reconstruction unit **214**, the loop filter **320** may be identical in function to the loop filter **220**, and the decoded picture buffer **330** may be identical in function to the decoded picture buffer **230**. Therefore, the explanations provided for the respective units and functions of the video **20** encoder apply correspondingly to the respective units and functions of the video decoder **30**.

Entropy Decoding

[0134] The entropy decoding unit **304** is configured to parse the bitstream **21** (or in general encoded picture data **21**) and perform, for example, entropy decoding to the encoded picture data **21** to obtain, e.g., quantized coefficients **309** and/or decoded coding parameters (not shown in FIG. 3), e.g. any or all of inter prediction parameters (e.g. reference picture index and motion vector), intra prediction parameter (e.g. intra prediction mode or index), transform parameters, quantization parameters, loop filter parameters, and/or other syntax elements. Entropy decoding unit **304** maybe configured to apply the decoding algorithms or schemes corresponding to the encoding schemes as described with regard to the entropy encoding unit **270** of the encoder **20**. Entropy decoding unit **304** may be further configured to provide inter prediction parameters, intra prediction parameter and/or other syntax elements to the mode selection unit **360** and other parameters to other units

of the decoder 30. Video decoder 30 may receive the syntax elements at the video slice level and/or the video block level.

Inverse Quantization

[0135] The inverse quantization unit 310 may be configured to receive quantization parameters (QP) (or in general information related to the inverse quantization) and quantized coefficients from the encoded picture data 21 (e.g. by parsing and/or decoding, e.g. by entropy decoding unit 304) and to apply based on the quantization parameters an inverse quantization on the decoded quantized coefficients 309 to obtain dequantized coefficients 311, which may also be referred to as transform coefficients 311. The inverse quantization process may include use of a quantization parameter determined by video encoder 20 for each video block in the video slice to determine a degree of quantization and, likewise, a degree of inverse quantization that should be applied.

Inverse Transform

[0136] Inverse transform processing unit 312 may be configured to receive dequantized coefficients 311, also referred to as transform coefficients 311, and to apply a transform to the dequantized coefficients 311 in order to obtain reconstructed residual blocks 213 in the sample domain. The reconstructed residual blocks 213 may also be referred to as transform blocks 313. The transform may be an inverse transform, e.g., an inverse DCT, an inverse DST, an inverse integer transform, or a conceptually similar inverse transform process. The inverse transform processing unit 312 may be further configured to receive transform parameters or corresponding information from the encoded picture data 21 (e.g. by parsing and/or decoding, e.g. by entropy decoding unit 304) to determine the transform to be applied to the dequantized coefficients 311.

Reconstruction

[0137] The reconstruction unit 314 (e.g. adder or summer 314) may be configured to add the reconstructed residual block 313, to the prediction block 365 to obtain a reconstructed block 315 in the sample domain, e.g. by adding the sample values of the reconstructed residual block 313 and the sample values of the prediction block 365.

Filtering

[0138] The loop filter unit 320 (either in the coding loop or after the coding loop) is configured to filter the reconstructed block 315 to obtain a filtered block 321, e.g. to smooth pixel transitions, or otherwise improve the video quality. The loop filter unit 320 may comprise one or more loop filters such as a de-blocking filter, a sample-adaptive offset (SAO) filter or one or more other filters, e.g. a bilateral filter, an adaptive loop filter (ALF), a sharpening, a smoothing filters or a collaborative filters, or any combination thereof. Although the loop filter unit 320 is shown in FIG. 3 as being an in loop filter, in other configurations, the loop filter unit 320 may be implemented as a post loop filter.

Decoded Picture Buffer

[0139] The decoded video blocks 321 of a picture are then stored in decoded picture buffer 330, which stores the

decoded pictures 331 as reference pictures for subsequent motion compensation for other pictures and/or for output respectively display.

[0140] The decoder 30 is configured to output the decoded picture 311, e.g. via output 312, for presentation or viewing to a user.

Prediction

[0141] The inter prediction unit 344 may be identical to the inter prediction unit 244 (in particular to the motion compensation unit) and the intra prediction unit 354 may be identical to the inter prediction unit 254 in function, and performs split or partitioning decisions and prediction based on the partitioning and/or prediction parameters or respective information received from the encoded picture data 21 (e.g. by parsing and/or decoding, e.g. by entropy decoding unit 304). Mode selection unit 360 may be configured to perform the prediction (intra or inter prediction) per block based on reconstructed pictures, blocks or respective samples (filtered or unfiltered) to obtain the prediction block 365.

[0142] When the video slice is coded as an intra coded (I) slice, intra prediction unit 354 of mode selection unit 360 is configured to generate prediction block 365 for a picture block of the current video slice based on a signaled intra prediction mode and data from previously decoded blocks of the current picture. When the video picture is coded as an inter coded (i.e., B, or P) slice, inter prediction unit 344 (e.g. motion compensation unit) of mode selection unit 360 is configured to produce prediction blocks 365 for a video block of the current video slice based on the motion vectors and other syntax elements received from entropy decoding unit 304. For inter prediction, the prediction blocks may be produced from one of the reference pictures within one of the reference picture lists. Video decoder 30 may construct the reference frame lists, List 0 and List 1, using default construction techniques based on reference pictures stored in DPB 330.

[0143] Mode selection unit 360 is configured to determine the prediction information for a video block of the current video slice by parsing the motion vectors and other syntax elements, and uses the prediction information to produce the prediction blocks for the current video block being decoded. For example, the mode selection unit 360 uses some of the received syntax elements to determine a prediction mode (e.g., intra or inter prediction) used to code the video blocks of the video slice, an inter prediction slice type (e.g., B slice, P slice, or GPB slice), construction information for one or more of the reference picture lists for the slice, motion vectors for each inter encoded video block of the slice, inter prediction status for each inter coded video block of the slice, and other information to decode the video blocks in the current video slice.

[0144] Other variations of the video decoder 30 can be used to decode the encoded picture data 21. For example, the decoder 30 can produce the output video stream without the loop filtering unit 320. For example, a non-transform based decoder 30 can inverse-quantize the residual signal directly without the inverse-transform processing unit 312 for certain blocks or frames. In another implementation, the video decoder 30 can have the inverse-quantization unit 310 and the inverse-transform processing unit 312 combined into a single unit.

[0145] It should be understood that, in the encoder 20 and the decoder 30, a processing result of a current operation may be further processed and then output to the next operation. For example, after interpolation filtering, motion vector derivation or loop filtering, a further operation, such as Clip or shift, may be performed on the processing result of the interpolation filtering, motion vector derivation or loop filtering.

[0146] It should be noted that further operations may be applied to the derived motion vectors of current block (including but not limit to control point motion vectors of affine mode, sub-block motion vectors in affine, planar, ATMVP modes, temporal motion vectors, and so on). For example, the value of motion vector is constrained to a predefined range according to its representing bit. If the representing bit of motion vector is bitDepth, then the range is $-2^{(bitDepth-1)} \sim 2^{(bitDepth-1)} - 1$, where “A” means exponentiation. For example, if bitDepth is set equal to 16, the range is $-32768 \sim 32767$; if bitDepth is set equal to 18, the range is $-131072 \sim 131071$. Here provides two methods for constraining the motion vector.

Method 1: Remove the Overflow MSB (Most Significant Bit) by Flowing Operations

$$ux = (mvx + 2^{bitDepth}) \% 2^{bitDepth} \quad (1)$$

$$mvx = (ux >= 2^{bitDepth-1}) ? (ux - 2^{bitDepth}) : ux \quad (2)$$

$$uy = (mvy + 2^{bitDepth}) \% 2^{bitDepth} \quad (3)$$

$$mvy = (uy >= 2^{bitDepth-1}) ? (uy - 2^{bitDepth}) : uy \quad (4)$$

[0147] For example, if the value of mvx is -32769 , after applying formula (1) and (2), the resulting value is 32767. In computer system, decimal numbers are stored as two's complement. The two's complement of -32769 is 1,0111, 1111, 1111, 1111 (17 bits), then the MSB is discarded, so the resulting two's complement is 0111, 1111, 1111, 1111 (decimal number is 32767), which is same as the output by applying formula (1) and (2).

$$ux = (mvpx + mvd_x + 2^{bitDepth}) \% 2^{bitDepth} \quad (5)$$

$$mvx = (ux >= 2^{bitDepth-1}) ? (ux - 2^{bitDepth}) : ux \quad (6)$$

$$uy = (mvpy + mvd_y + 2^{bitDepth}) \% 2^{bitDepth} \quad (7)$$

$$mvy = (uy >= 2^{bitDepth-1}) ? (uy - 2^{bitDepth}) : uy \quad (8)$$

The operations may be applied during the sum of mvp and mvd, as shown in formula (5) to (8).

Method 2: Remove the Overflow MSB by Clipping the Value

$$vx = \text{Clip3}(-2^{bitDepth-1}, 2^{bitDepth-1} - 1, vx)$$

$$vy = \text{Clip3}(-2^{bitDepth-1}, 2^{bitDepth-1} - 1, vy)$$

[0148] where the definition of function Clip3 is as follow:

$$\text{Clip3}(x, y, z) = \begin{cases} x & ; \quad z < x \\ y & ; \quad z > y \\ z & ; \quad \text{otherwise} \end{cases}$$

[0149] FIG. 4 is a schematic diagram of a video coding device 400 according to an embodiment of the disclosure. The video coding device 400 is suitable for implementing the disclosed embodiments as described herein. In an embodiment, the video coding device 400 may be a decoder such as video decoder 30 of FIG. 1A or an encoder such as video encoder 20 of FIG. 1A.

[0150] The video coding device 400 comprises ingress ports 410 (or input ports 410) and receiver units (Rx) 420 for receiving data; a processor, logic unit, or central processing unit (CPU) 430 to process the data; transmitter units (Tx) 440 and egress ports 450 (or output ports 450) for transmitting the data; and a memory 460 for storing the data. The video coding device 400 may also comprise optical-to-electrical (OE) components and electrical-to-optical (EO) components coupled to the ingress ports 410, the receiver units 420, the transmitter units 440, and the egress ports 450 for egress or ingress of optical or electrical signals.

[0151] The processor 430 is implemented by hardware and software. The processor 430 may be implemented as one or more CPU chips, cores (e.g., as a multi-core processor), FPGAs, ASICs, and DSPs. The processor 430 is in communication with the ingress ports 410, receiver units 420, transmitter units 440, egress ports 450, and memory 460. The processor 430 comprises a coding module 470. The coding module 470 implements the disclosed embodiments described above. For instance, the coding module 470 implements, processes, prepares, or provides the various coding operations. The inclusion of the coding module 470 therefore provides a substantial improvement to the functionality of the video coding device 400 and effects a transformation of the video coding device 400 to a different state. Alternatively, the coding module 470 is implemented as instructions stored in the memory 460 and executed by the processor 430.

[0152] The memory 460 may comprise one or more disks, tape drives, and solid-state drives and may be used as an over-flow data storage device, to store programs when such programs are selected for execution, and to store instructions and data that are read during program execution. The memory 460 may be, for example, volatile and/or non-volatile and may be a read-only memory (ROM), random access memory (RAM), ternary content-addressable memory (TCAM), and/or static random-access memory (SRAM).

[0153] FIG. 5 is a simplified block diagram of an apparatus 500 that may be used as either or both of the source device 12 and the destination device 14 from FIG. 1 according to an embodiment.

[0154] A processor 502 in the apparatus 500 can be a central processing unit. Alternatively, the processor 502 can be any other type of device, or multiple devices, capable of manipulating or processing information now-existing or hereafter developed. Although the disclosed implementations can be practiced with a single processor as shown, e.g.,

the processor **502**, advantages in speed and efficiency can be achieved using more than one processor.

[0155] A memory **504** in the apparatus **500** can be a read only memory (ROM) device or a random access memory (RAM) device in an implementation. Any other suitable type of storage device can be used as the memory **504**. The memory **504** can include code and data **506** that is accessed by the processor **502** using a bus **512**. The memory **504** can further include an operating system **508** and application programs **510**, the application programs **510** including at least one program that permits the processor **502** to perform the methods described here. For example, the application programs **510** can include applications **1** through **N**, which further include a video coding application that performs the methods described here.

[0156] The apparatus **500** can also include one or more output devices, such as a display **518**. The display **518** may be, in one example, a touch sensitive display that combines a display with a touch sensitive element that is operable to sense touch inputs. The display **518** can be coupled to the processor **502** via the bus **512**.

[0157] Although depicted here as a single bus, the bus **512** of the apparatus **500** can be composed of multiple buses. Further, the secondary storage **514** can be directly coupled to the other components of the apparatus **500** or can be accessed via a network and can comprise a single integrated unit such as a memory card or multiple units such as multiple memory cards. The apparatus **500** can thus be implemented in a wide variety of configurations.

[0158] In an example about Merge Candidate List Construction according to ITU-T H.265, a merge candidate list is constructed based on the following candidates:

- [0159] 1. up to four spatial candidates that are derived from five spatial neighboring blocks,
- [0160] 2. one temporal candidate derived from two temporal, co-located blocks,
- [0161] 3. additional candidates including combined bi-predictive candidates and,
- [0162] 4. zero motion vector candidates.

Spatial Candidates

[0163] The motion information of spatial neighbor blocks are first added into the merge candidate list (in an example, the merge candidate list may be an empty list before the first motion vector is added into the merge candidate list) as motion information candidates. Here, the neighboring blocks that are considered to be inserted in the merge list are illustrated in FIG. 6b. For inter-prediction block merging, up to four candidates are inserted in the merge list by sequentially checking A1, B1, B0, A0 and B2, in that order.

[0164] Motion information may contains all motion data including the information whether one or two reference picture lists are used as well as a reference index and a motion vector for each reference picture list.

[0165] In an example, after checking whether a neighboring block is available and contains motion information, some additional redundancy checks are performed before taking all the motion data of the neighboring block as a motion information candidate. These redundancy checks can be divided into two categories for two different purposes:

- [0166] Category 1, avoid having candidates with redundant motion data in the list,

- [0167] Category 2, prevent merging two partitions that could be expressed by other means which would create redundant syntax.

Temporal Candidates

[0168] The FIG. 6a illustrated the coordinates of the blocks where temporal motion information candidates are retrieved from. Collocated block is the block that has the same $-x$, $-y$ coordinates of the current block, but is on a different picture (one of the reference pictures). The temporal motion information candidates are added to the merge list if the list is not full (in an example, the merge list is not full when a quantity of candidates in the merge list is smaller than a threshold, for example, the threshold may be 4, 5, 6, and so on).

Generated Candidates

[0169] After insertion of spatial and temporal motion information candidates, if the merge list is still not full, generated candidates are added to fill the list. The list size is indicated in the sequence parameter set and is fixed throughout the whole coded video sequence.

[0170] The merge list construction process in ITU-T H.265 and in VVC outputs a list of motion information candidates. The merge list construction process of VVC is described in the section “8.3.2.2 Derivation process for luma motion vectors for merge mode” section of the document JVET-L1001_v2 Versatile Video Coding (Draft 3), which is publicly available under <http://phenix.it-sudparis.eu/jvet/>. The term motion information refers to the motion data that is necessary to perform motion compensated prediction process. The motion information usually refers to the following information:

- [0171] Whether the block applies uni-prediction or bi-prediction
- [0172] The ID of the reference picture that is used in the prediction. (2 IDs if block applies bi-prediction).
- [0173] Motion vector (2 motion vectors if block is bi-predicted)
- [0174] Additional information

[0175] In VVC and H.265, the list of candidates that are output of the merge list construction include N candidate motion information. The number N is typically included in the bitstream and can be a positive integer number such as 5, 6 etc. The candidates that are included in the constructed merge list might include uni-prediction information or bi-prediction information. This means that the candidate that is selected from the merge list might indicate a bi-prediction operation.

Bi-Prediction

[0176] A special mode of inter prediction is called “bi-prediction”, where 2 motion vectors are used to predict a block. The motion vectors can point to same or different reference pictures, where a reference picture can be indicated by a reference picture list ID and a reference picture index. For example a first motion vector might point to a first picture in the reference picture list L0 and a second motion vector might point to a first picture in the reference picture list L1. Two reference picture lists (for example, L0 and L1) might be maintained and the picture pointed by the first motion vector is selected from list L0 and the picture pointed by the second motion vector is selected from list L1.

[0177] In an example, if a motion information indicates bi-prediction, then the motion information includes two parts:

[0178] L0 part: A motion vector and a reference picture index that points to an entry in the reference picture list L0.

[0179] L1 part: A motion vector and a reference picture index that points to an entry in the reference picture list L1.

[0180] Picture Order Count (POC): A variable that is associated with each picture, uniquely identifies the associated picture among all pictures in the CVS (Coded Video Sequence), and, when the associated picture is to be output from the decoded picture buffer, indicates the position of the associated picture in output order relative to the output order positions of the other pictures in the same CVS that are to be output from the decoded picture buffer.

[0181] Each of the reference picture lists L0 and L1 might include one or more reference pictures each of which is identified with a POC. The association with each reference index and the POC value might be signaled in the bitstream. As an example the L0 and L1 reference picture lists might include the following reference pictures:

Reference Picture List	Reference Index	POC
L0	0	12
L0	1	13
L1	0	13
L1	1	14

[0182] In the example above, the first entry (indicated by reference index 0) in reference picture list L1 is the reference picture with POC value 13. The second entry (indicated by reference index 1) in reference picture list L1 is the reference picture with POC value 14.

Triangular Prediction Mode

[0183] The concept of the triangular prediction mode is to introduce a triangular partition for motion compensated prediction. As an example shown in FIG. 7, two triangular prediction units are used for a CU, in either diagonal or inverse diagonal direction. Each triangular prediction unit in the CU is inter-predicted, using uni-prediction motion vector and reference frame index which are derived from a uni-prediction candidate list. An adaptive weighting process is performed to the diagonal edge after the samples that are associated with each triangular prediction unit have been predicted for example by motion-compensation or intra-picture prediction. Then, the transform and quantization process are applied to the whole CU. It is noted that this mode is only applied to skip mode and merge mode.

[0184] In the triangular prediction mode, a block is split into two triangular parts (as in FIG. 7), and each part can be predicted using one motion vector. A motion vector which is used to predict one triangle part (denoted with PU1) can be different from a motion vector which is used to predict the other triangle part (denoted with PU2). In an example, it is noted that each part can be predicted only using a single motion vector (uni-prediction), in order to reduce the complexity of performing the triangle prediction mode. In other words, the PU1 and PU2 may not be predicted using bi-prediction, which comprises two motion vector.

Sub-Block Prediction Mode

[0185] Triangular prediction mode is a special case of sub-block prediction, where a block is divided into two blocks. In the above example, two block division directions are illustrated (45 degree and 135 degree partitions.) It is noted that other partition angles and partition proportions are also possible (examples in FIG. 8).

[0186] In some examples, the block is split into 2 sub-block parts and each part is required to apply uni-prediction. Sub-block prediction represents a generalized version of triangular prediction.

[0187] In an example, according to use sub-block partition mode, the following operations are applied to obtain prediction samples:

[0188] Operation 1: divide a coding block into 2 sub-blocks, according to a geometric model. This model may result in the splitting of the block by a separation line (for example, a straight line) as exemplified in FIG. 9-12.

[0189] It can be understood that, the result of Operation 1 may be just concept. For example, there is no express concept “sub-block 1” or “sub-block 2” after the Operation 1; but generating or obtaining parameters refers to the separation line. Samples may belong to different sub-blocks according to the parameters referring to the separation line.

[0190] In this operation, according to a geometric model, samples in a coding block are considered as located in two sub-blocks. Sub-block A or sub-block B comprises a part (but not all) of samples in the current coding block. Sub-block A or sub-block B may be represented according to the sign of a sample_dist of each samples. The sample_dist may be obtained according to the examples and embodiments in the other paragraphs.

[0191] Operation 2: obtain a first prediction mode for a first sub-block and a second prediction mode for a second sub-block. In an example, the first prediction mode is not identical to the second prediction mode. In an example, a prediction mode (first prediction mode or second prediction mode) may be an inter prediction mode, information for an inter prediction mode may comprises a reference picture index and a motion vector. In another example, the prediction mode may be an intra-prediction mode, information for an intra the prediction mode may comprises an intra prediction mode index.

[0192] Operation 3: Obtain first prediction values and second prediction values, using the first prediction mode and second prediction mode, respectively.

[0193] Operation 4: obtain the combined values of prediction samples according to the combination of the first prediction values and the second prediction values, according to the division which is disclosed in Operation 1. The process of combining samples from the first prediction and the second prediction to obtain the combined values of prediction samples of a block may comprise filtering operations, masking operations or copying of samples.

[0194] In an example, in operation 1, a coding block is divided into 2 sub-blocks in various ways. FIG. 9 shows an example about partition of a coding block, a separation line 1250 divides the block into 2 sub-blocks. In order to

describe the line **1250**, two parameters are signaled, one parameter is angle α **1210** and the other parameter is distance $dist$ **1230**.

[0195] In some embodiments, the angle, as indicated in FIG. 9, is measured between the x-axis and the separation line, whereas the distance is measured by the length of the vector which is perpendicular to the separation line and passing through the center of the current block.

[0196] In another example, FIG. 10 shows an alternative way of representing the separation line, where the examples of the angle and the distance are different with the examples showed in FIG. 9.

[0197] In some examples, in operation 4, the division disclosed in Operation 1 is used for the combination of the first prediction values and second prediction values, to obtain the final prediction values.

[0198] In an example, a blending operation is applied in operation 4 in order to eliminate any artefacts (edgy or jugged appearance along the separation line). The blending operation can be described as a filtering operation along the separation line.

[0199] In the encoder side, a separation line (the parameters defining the line, e.g. the angular and distance) is determined based on a rate-distortion based cost function. The determined line parameters are encoded into the bitstream. In the decoder side, the line parameters are decoded (obtained) according to the bitstream. In the decoder side, the line parameters are used for the combination of the first prediction values and second prediction values, to obtain the final prediction values. The Operation 1 does not require there are two sub-coding block concepts, the decoder can use a coding block and the line parameters to represent two sub-coding block concepts.

[0200] In an example, in the case of 3 video channels, which comprise a luminance component and two chrominance components, a first prediction and a second prediction is generated for each channel.

[0201] As there are many possibilities of dividing a coding block into 2 sub-blocks, the signaling (coding) of the division (or partition) requires many bits. Since the angle and distance values can have many different values, which require too much side information to be signaled in a bitstream, a quantization scheme is applied to the angle and distance side information to improve coding efficiency.

[0202] Embodiments of the present disclosure are about signaling of the partitioning parameters, for example, a quantized angle parameter αIdx and a quantized distance parameter $distIdx$.

[0203] In an example, in a quantization scheme, an angle value and a distance value can be quantized by a linear, uniform quantizer according to

$$\alphaIdx = \text{Round}(\alpha/\Delta\alpha),$$

$$distIdx = \text{Round}(dist/\Delta dist),$$

wherein the variables $\Delta\alpha$ represents quantization step sizes for the angular parameter a separation line and $\Delta dist$ represents quantization step sizes for the distance parameter of the separation line. Accordingly, an angle α and a distance $dist$ may be reconstructed using values of αIdx and $distIdx$ that obtained from a bitstream as

$$\alpha = \alphaIdx * \Delta\alpha,$$

$$dist = distIdx * \Delta dist.$$

[0204] In an example, if an angular quantization step size is equal to $\Delta\alpha=11.25^\circ$, the value of αIdx may range from 0 to 31 (includes 0 and 31) to cover the allowed angles in the Euclidean plane. In another example, a value of the angular quantization step size is equal to $\Delta\alpha=22.5^\circ$, thereby the value of αIdx may range from 0 to 15 (includes 0 and 15).

[0205] In some embodiments, a method is disclosed to quantize the distance information in dependence of the angular, in the case that a linear and uniform quantizer is used to quantize the angular, meaning that the value of $\Delta\alpha$ is constant for a given coding (decoding or encoding) block.

[0206] The goal of the disclosed quantization scheme is to evenly distribute the allowed separation lines for a given coding block, which can have different sizes and aspect ratios. The evenly distributed space of quantized angle and distance parameters improves the coding efficiency of non-rectangular components and allows a compact representation of the separation line parameters.

[0207] In some embodiments, a method of partitioning a rectangular coding block by a straight line, where the straight line is parametrized by a pair of parameters, representing a quantized angle and a quantized distance value, and the quantized distance value is derived according to a quantization process, depending on the value of the angle and the aspect ratio of the coding block.

[0208] In an example, a distance may be quantized in such a manner, that a given value range of $distIdx$ shall be satisfied, for example, a value range from 0 to 3 (includes 0 and 3). In another example, the distance may be quantized for a given block in such a manner, that the separation lines for given pairs of αIdx and $distIdx$ values are evenly distributed and a separation line shall never lie outside of the area of the given coding block.

[0209] In a first operation, a maximum distance p_{max} can be derived depending on the angle, such that all separation lines with a distance value $0 < dist < p_{max}$ are confined to the coding block (e.g. they are intersecting the boundary of the coding block). This is exemplified in FIG. 15, for a coding block of size 16×4 luminance samples.

[0210] In an example, the maximum distance p_{max} can be derived as a function, depending on an angle αR and a size of a coding block according to:

$$p_{max}(\alpha R, w, h) = \cos(\alpha R) \left(\frac{h}{2 \tan\left(\frac{\pi}{2} - \alpha R\right)} + \frac{w}{2} \right),$$

$$0 \leq \alpha R < \frac{\pi}{2},$$

where αR is the angle previously denoted as α in terms of radiant units, w is the width of the block and h is the height of the block, in terms of luminance samples. In the example, the value of the angle dependent distance quantization step size $\Delta dist(\alpha)$ can be derived as

$$\Delta dist(alphaR, w, h) = \rho_{max}(alphaR, w, h)/N,$$

where $N-1$ is the maximum value of distanceIdx. This is exemplified in FIG. 16 with values of $\Delta angle=22.5^\circ$ and $N=4$.

In another example, the value of the angle dependent distance quantization step size can be derived as

$$\Delta dist(alphaR, w, h) = (\rho_{max}(alphaR, w, h) - \rho_{th})/N,$$

where ρ_{th} is a parameter used to modify the spacing between the separation lines for a coding block. In an example, the value of ρ_{th} is set to $\rho_{th}=1.5$.

In another example, the maximum distance ρ_{max} can be derived as a function, depending on an angle and a size of a coding block according to:

$$\rho_{max}(alphaR, w, h) = \frac{1}{2\sqrt{\left(\frac{\cos(alphaR)}{w}\right)^2 + \left(\frac{\sin(alphaR)}{h}\right)^2}},$$

$$0 \leq alphaR < \frac{\pi}{2},$$

where $alphaR$ is the angle in terms of radiant units, w is the width of the block and h is the height of the block, in terms of luminance samples. This is exemplified in FIG. 17 with values of $\Delta angle=22.5^\circ$ and $N=4$.

[0211] In an example, values of $\Delta dist$ depending on the value of a parameter representing an angle, a parameter representing the width and a parameter representing a height can be stored in a pre-computed look-up table, in order to avoid the repeated computation of the $\Delta dist$ during the encoding or decoding process.

[0212] In an embodiment, values of $\Delta dist$ can be scaled and rounded, for purposes of using integer arithmetic according to

$$stepD = \text{Round}(\Delta dist * 2^{prec}),$$

where $stepD$ represents a linearly scaled distance step size and $prec$ is a variable used to adjust the precision of the scaling process. In an example, the value of $prec$ is set to $prec=7$.

In an example, it is further store pre-computed values of $stepD$ based on an aspect ratio denoted $whRatio$, the aspect ratio value depends on the width and height of a coding block. Further, pre-computed values of $stepD$ is stored based on an (normalized) angle value $angleN$, which is an index value that relates to angles in the first quadrant of the Euclidean plane (e.g. $0 \leq angleN * \Delta alpha \leq 90^\circ$). An example of such a look-up table where the aforementioned simplifications have been applied can be seen in FIG. 13.

[0213] In one embodiment, the following operations are applied to obtain prediction values for samples of a coding block.

[0214] Operation 1: For a sample in a current block (decoding block or encoding block), a sample distance (sample_dist) is calculated.

[0215] In some examples, the sample distance may represents the horizontal distance or the vertical distance, or a combination of vertical and horizontal distance, of the said sample to a separation line, the separation line is used to divide a coding block into two sub-blocks. The said sample is represented by coordinates (x,y) with respect to the top-left sample of the coding block. The sample coordinate sample (x, y) and the sample_dist are exemplified in FIG. 11 and FIG. 12. The sub-block is not necessarily rectangular, the sub-block may be triangular or trapezoid in shape.

[0216] In an example, a first parameter represents a quantized angle value (angleIdx) and a second parameter represents a quantized distance value (distanceIdx). The two parameters describe a line equation. In an example, the distance 1230 can be obtained according to distanceIdx (second parameter), and angle α 1210 can be obtained according to angleIdx (first parameter). The distance 1230 can be a distance of a separation line to the center of a coding block, and the angle can be an angle between the separation line and the horizontal (or equivalently the vertical) line passing through the center point of the coding block.

[0217] In a specific example, the value of angleIdx and the value of distanceIdx are obtained based on a partitioning shape of geometric partitioning merge mode (which is indicated by `merge_gpm_partition_idx`). For example, the relationship among these values are shown in table 1. It could be understood that, in the implementation, this relationship can be implemented according to table 1 or according to a function.

TABLE 1

		Specification of angleIdx and distanceIdx based on merge_gpm_partition_idx.															
		merge_gpm_partition_idx															
		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
angleIdx	0	0	2	2	2	2	3	3	3	3	4	4	4	4	5	5	
distanceIdx	1	3	0	1	2	3	0	1	2	3	0	1	2	3	0	1	
		merge_gpm_partition_idx															
		16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
angleIdx	5	5	8	8	11	11	11	11	12	12	12	12	13	13	13	13	
distanceIdx	2	3	1	3	0	1	2	3	0	1	2	3	0	1	2	3	

TABLE 1-continued

Specification of angleIdx and distanceIdx based on merge_gpm_partition_idx.		merge_gpm_partition_idx															
		32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
angleIdx		14	14	14	14	16	16	18	18	18	19	19	19	20	20	20	21
distanceIdx		0	1	2	3	1	3	1	2	3	1	2	3	1	2	3	1
		merge_gpm_partition_idx															
		48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
angleIdx		21	21	24	24	27	27	27	28	28	28	29	29	29	30	30	30
distanceIdx		2	3	1	3	1	2	3	1	2	3	1	2	3	1	2	3

[0218] In an example, in operation 1, a coding block is divided into 2 sub-blocks in various ways. FIG. 9 shows an example about partition of a coding block, the separation line **1250** divides the block into 2 sub-blocks. In order to describe the line **1250**, one angle parameter **alpha 1210** is signaled in a bitstream.

[0219] In some embodiments, the angle, as indicated in FIG. 9, is measured between the x-axis and the separation line, whereas the distance is measured by the length of the vector which is perpendicular to the separation line and passing through the center of the current block.

[0220] In another example, FIG. 10 shows an alternative way of representing the separation line, where the examples of the angle and the distance are different with the examples showed in FIG. 9.

[0221] Operation 2: The calculated sample_dist is used to calculate weighting factors, the weighting factors are used for the combination of a first prediction value and a second prediction value corresponding to the said sample. In an example, the weighting factors are denoted as sampleWeight1 and sampleWeight2, referring to the weight corresponding to the first prediction value and the weight corresponding to the second prediction value.

[0222] In one example, weighting factors are calculated according to the following functions,

$$weightIdxL = partFlip ? 32 + sample_dist : 32 - sample_dist;$$

$$wValue = Clip3(0, 8, (weightIdxL + 4) >> 3).$$

[0223] In this example, wValue is sampleWeight1, and 8-w Value is sampleWeight2. The variable partFlip is determined according to a value of angleIdx. In an example, partFlip=(angleIdx>=13 && angleIdx<=27)?0:1, or partFlip=(angleIdx>=13 && angleIdx<=27)?1:0.

[0224] Operation 3: The combined value of the prediction sample at the sample coordinate (x,y) is calculated according to the first prediction value at coordinate (x,y), the second prediction value at coordinate (x,y), sampleWeight1 and sampleWeight2.

[0225] In an example, the prediction sample value is derived as follows:

$$pbSamples[x][y] = Clip3(0, (1 << BitDepth) - 1, (predSamplesLA[x][y] * wValue + predSamplesLB[x][y] * (8 - wValue) + offset1) >> shift1).$$

Wherein bitDepth represents sample bit depth, variable shift1 is obtained according to the bitDepth, in an example, shift1=Max(5, 17-BitDepth); Variable offset1 is obtained according to the shift1, in an example, offset1=1<<(shift1-1), predSamplesLA and predSamplesLB are two (nCbW)×(nCbH) arrays.

[0226] In an embodiment, according to one example, the Operation 1 in the above embodiments may comprise the following operations:

[0227] Operation 1.1: obtain an index value of an angle parameter (alphaN or angleIdx) for the current block, a value of a width of the current block (W), a value of a height of the current block (H). The W and H are width and height of the current block in number of samples. For example, a coding block with width and height both equal to 8 is a square block that comprises 64 samples. In another example, the W and H are width and height of the current block, in number of luma samples. The index value of the angle parameter may be obtained according to the above description refer to table 1.

[0228] Operation 1.2: obtain a value of a ratio whRatio according to the value of W and the value of H, the value of whRatio represent a ratio between the width and height of the current coding block.

[0229] In an example, whRatio=H/W; or whRatio=W/H.

[0230] In another example, two variables nCbW and nCbH specify the width and the height of the current coding block, a variable cIdx specify a colour component index.

[0231] variables nW, nH and whRatio are derived as follows:

$$nW = (cIdx == 0) ? nCbW : nCbW * SubWidthC;$$

$$nH = (cIdx == 0) ? nCbH : nCbH * SubHeightC;$$

$$whRatio = nH/nW.$$

[0232] In an example, the variables SubWidthC and SubHeightC are specified in table 2, depending on the chroma format sampling structure, which is specified through chroma_format_idc(chroma_format_idc specifies the chroma sampling relative to the luma sampling) and separate_colour_plane_flag(separate_colour_plane_flag equal to 1 specifies that the three colour components of the 4:4:4 chroma format are coded separately. separate_colour_plane_flag equal to 0 specifies that the colour components are not coded separately. When separate_colour_plane_flag is not present, it is inferred to be equal to 0. When separate_colour_plane_flag is equal to 1, the coded picture consists of three separate components, each of which consists of coded samples of one colour plane (Y, Cb, or Cr) and uses the monochrome coding syntax. In this case, each colour plane is associated with a specific colour_plane_id value). Other values of chroma_format_idc, SubWidthC and SubHeightC may be specified in the future by ITU-T/ISO/IEC.

TABLE 2

Sub WidthC and SubHeightC values derived from chroma_format_idc and separate_colour_plane_flag				
chroma_format_idc	separate_colour_plane_flag	Chroma format	SubWidthC	SubHeightC
0	0	Monochrome	1	1
1	0	4:2:0	2	2
2	0	4:2:2	2	1
3	0	4:4:4	1	1
3	1	4:4:4	1	1

[0233] In monochrome sampling there is only one sample array, which is nominally considered the luma array.

[0234] In 4:2:0 sampling, each of the two chroma arrays has half the height and half the width of the luma array.

[0235] In 4:2:2 sampling, each of the two chroma arrays has the same height and half the width of the luma array.

[0236] In 4:4:4 sampling, depending on the value of separate_colour_plane_flag, the following applies:

[0237] If separate_colour_plane_flag is equal to 0, each of the two chroma arrays has the same height and width as the luma array.

[0238] Otherwise (separate_colour_plane_flag is equal to 1), the three colour planes are separately processed as monochrome sampled pictures.

[0239] Operation 1.3: Obtain a stepD value according to a lookup table, the value of the alpha and the value of the whRatio, in an example, as showed in FIG. 13, the value of the alpha (alphaN or angleIdx) and the value of the whRatio are used as index values of the lookup table. The stepD value can also be obtained according to a function, wherein the value of the alpha (alphaN or angleIdx) and the value of the whRatio are the input to the function and the stepD value is the output of the function. In an example, the result of the function is similar or same as the lookup table. In an example, the stepD value represents a quantization step size for the sample distance calculation process.

[0240] In another example, the function may be represented by $\text{stepD} = (\text{angleIdx} \% 16 == 8) ? ((\text{angleIdx} \% 16 != 0 \&\& \text{whRatio} > 0)) ? 0 : 1$.

[0241] Operation 1.4: a value of the sample_dist is calculated according to the stepD value.

[0242] In an example, for calculating sample distance (sample_dist), first, variables offsetX and offsetY are derived as follows:

If stepD is equal to 0, the following applies:

$$\text{offsetX} = (-nW) >> 1,$$

$$\text{offsetY} = ((-nH) >> 1) + (\text{angleIdx} < 16 ? (\text{distanceIdx} * nH) >> 3 : -((\text{distanceIdx} * nH) >> 3));$$

Otherwise (stepD is equal to 1), the following applies:

$$\begin{aligned} \text{offsetX} &= ((-nW) >> 1) + (\text{angleIdx} < 16 ? (\text{distanceIdx} * nW) >> 3 : -((\text{distanceIdx} * nW) >> 3)); \\ \text{offsetY} &= (-nH) >> 1; \end{aligned}$$

[0243] The variables xL and yL are derived as follows:

$$xL = (\text{cIdx} == 0) ? x : x * \text{SubWidthC}$$

$$yL = (\text{cIdx} == 0) ? y : y * \text{SubHeightC}$$

$$\begin{aligned} \text{sample_dist} &= (((xL + \text{offsetX}) < 1) + 1) * \text{disLut}[\text{displacementX}] + \\ &(((yL + \text{offsetY}) < 1) + 1) * \text{disLut}[\text{displacementY}]. \end{aligned}$$

[0244] The variables displacementX and displacementY are derived as follows:

[0245] hwRatio=cbHeight/cbWidth;

[0246] displacementX=angleIdx;

[0247] displacementY=(angleIdx+8) % 32.

[0248] The array disLut specified in table 3 as follows:

TABLE 3

Specification of the geometric partitioning distance array disLut.														
	idx													
	0	2	3	4	5	6	8	10	11	12	13	14		
disLut[idx]	8	8	8	4	4	2	0	-2	-4	-4	-8	-8		
	idx													
	16	18	19	20	21	22	24	26	27	28	29	30		
disLut[idx]	-8	-8	-8	-4	-4	-2	0	2	4	4	8	8		

[0249] In another embodiment, in an example, the Operation 1 in the above embodiments may comprise the following operations:

[0250] Operation 1.1: obtain a value of an angle parameter (alphaN or angleIdx) for the current block, a value of a distance index (distanceIdx), a value of a width of the current block (W), a value of a height of the current block (H). The W and H are width and height of the current block in number of samples. For example, a coding block with width and height both equal to 8 is a square block that comprises 64 samples. In another example, the W and H are width and height of the current block, in number of luma

samples. The index value of the angle parameter may be obtained according to the above description refer to table 1.

[0251] Operation 1.2: obtain a value of a ratio whRatio according to the value of W and the value of H, the value of whRatio represent a ratio between the width and height of the current coding block.

[0252] In an example, whRatio=H/W; or whRatio=W/H.

[0253] In another example, two variables nCbW and nCbH specify the width and the height of the current coding block, a variable cIdx specify a colour component index.

[0254] Variables nW, nH and whRatio are derived as follows:

$$\begin{aligned} nW &= (cIdx == 0) ? nCbW : nCbW * SubWidthC \\ nH &= (cIdx == 0) ? nCbH : nCbH * SubHeightC \\ whRatio &= nH / nW. \end{aligned}$$

[0255] Operation 1.3: Obtain a stepD value according to a lookup table, the value of the alpha and the value of the whRatio, in an example, as showed in FIG. 13, the value of the alphaN and the value of the whRatio are used as index values of the lookup table. In an example, the stepD value represents a quantization step size for the sample distance calculation process. It could be understand that, the stepD value also could be obtained according to a function, wherein the value of the alpha (alphaN or angleIdx) and the value of the whRatio are the input to the function and the step D value is the output of the function. In an example, the result of the function is similar or same as the lookup table. In an example, the stepD value represents a quantization step size for the sample distance calculation process.

[0256] In another example, the function may be represented by $stepD = (angleIdx \% 16 == 8) ? ((angleIdx \% 16 != 0 \&\& whRatio > 0) ? 0 : 1)$.

[0257] Operation 1.4: a value of the sample_dist is calculated according to the stepD value, the value of the distanceIdx, the value of the angle (alphaN or angleIdx), the value of W and the value of H.

[0258] In an example, for calculating sample distance (sample_dist), first, variables offsetX and offsetY are derived as follows:

If stepD is equal to 0, the following applies:

$$\begin{aligned} offsetX &= (-nW) >> 1, \\ offsetY &= ((-nH) >> 1) + \\ &\quad (angleIdx < 16 ? (distanceIdx * nH) >> 3 : -((distanceIdx * nH) >> 3)); \end{aligned}$$

Otherwise (stepD is equal to 1), the following applies:

$$\begin{aligned} offsetX &= ((-nW) >> 1) + \\ &\quad (angleIdx < 16 ? (distanceIdx * nW) >> 3 : -((distanceIdx * nW) >> 3)); \\ offsetY &= (-nH) > 1; \end{aligned}$$

[0259] The variables xL and yL are derived as follows:

$$\begin{aligned} xL &= (cIdx == 0) ? x : x * SubWidthC \\ yL &= (cIdx == 0) ? y : y * SubHeightC \\ sample_dist &= (((xL + offsetX) << 1) + 1) * disLut[displacementX] + \\ &\quad (((yL + offsetY) << 1) + 1) * disLut[displacementY]. \end{aligned}$$

[0260] The variables displacementX and displacementY are derived as follows:

$$\begin{aligned} hwRatio &= cbHeight / cbWidth; \\ displacementX &= angleIdx; \\ displacementY &= (angleIdx + 8) \% 32. \end{aligned}$$

The array disLut is specified in the above table 3.

[0261] In one example, a value of whRatio is obtained using the formula:

$$\begin{aligned} whRatio &= (wIdx \geq hIdx) ? wIdx - hIdx : hIdx - wIdx; \\ \text{where a value of } wIdx &\text{ is equal to } \log_2(W) - 3, \text{ and} \\ \text{a value of } hIdx &\text{ is equal to } \log_2(H) - 3. \end{aligned}$$

[0262] In another example, a value of whRatio is calculated as $whRatio = (W \geq H) ? W/H : H/W$.

[0263] In embodiments of the present disclosure, a value of angle alpha can be obtained from the bitstream (in the decoder). In one example, a value range for the angle is a quantized value range between 0 and 31 (includes 0 and 31), denoted as angleIdx. In one example, a quantized angle value only take 32 different distinct values (therefore a value between 0-31 is enough to represent which angle value is selected). In another example, a value range of the angle value may be between 0-15, which means that 16 different quantized angle values can be selected. It is noted that in general an angle value may be an integer value greater can zero.

[0264] In one example, a value of alphaN is an index value which is obtained from the bitstream, or a value of alpha is calculated based on a value of an indicator which is obtained from the bitstream. For example, a value of alphaN may be calculated as according to the formula:

$$\begin{aligned} \alpha N &= angleIdx && \text{if } angleIdx \geq 0 \&\& angleIdx \leq Lim, \text{ or} \\ \alpha N &= 2 * Lim - angleIdx && \text{if } angleIdx > Lim \&\& \\ &&& angleIdx \leq 2 * Lim, \text{ or} \\ \alpha N &= angleIdx - 2 * Lim && \text{if } angleIdx > 2 * Lim \&\& \\ &&& angleIdx \leq 3 * Lim, \text{ or} \\ \alpha N &= 4 * Lim - angleIdx && \text{otherwise.} \end{aligned}$$

whereas angleIdx is a value of an indicator which is obtained from the bitstream. A value of the angleIdx might be an integer value, a value range for angleIdx is between 0-31 (includes 0 and 31).

[0265] In another example, the value αN can be calculated according to one of the following formula:

$$\begin{aligned}\alpha N &= \text{angleIdx}, \text{ or} \\ \alpha N &= 2 * \text{Lim} - \text{angleIdx}, \text{ or} \\ \alpha N &= \text{angleIdx} - 2 * \text{Lim}, \text{ or} \\ \alpha N &= 4 * \text{Lim} - \text{angleIdx}.\end{aligned}$$

[0266] In the above formulas, Lim is a predefined constant value that is related to the number of quantized angle values. For example, if there are 32 different quantized angle values, the Lim may be equal to 8 ($4 * \text{Lim}$ is equal to 32). In another example, Lim can be 4, which corresponds to a total of 16 different quantized angle values.

[0267] In one embodiment, a value of the sample_dist is obtained according to the formula:

$$\text{sample_dist} = ((2 * x) - W + K) * f1(\alpha) + ((2 * y) - H + K) * f2(\alpha) - \text{distanceIdx} * \text{stepD} * \text{scaleStep}, \text{ where:}$$

[0268] $f1()$ and $f2()$ are functions, and α is an input for the functions. In an example, $f1()$ and $f2()$ are implemented as lookup tables, a value of the α is an index value for the lookup tables.

[0269] a value of scaleStep is equal to either a value of the width of a coding (decoding or encoding) block or a value of the height of the coding block.

[0270] x and y are the coordinates of the said sample with respect to the top-left sample of the coding block.

[0271] K is an integer offset value. In an example, a value for K is 1.

[0272] In one embodiment, functions $f1()$ and $f2()$ are implemented as lookup tables. In an example, functions $f1()$ and $f2()$ represent an incremental change in the sample_dist value with respect to changing x and y values. In some examples, $f1(\text{index})$ represents the sample_dist value is changed with one unit increase in the value of x (unit might be increase is equal to 1), whereas the $f2(\text{index})$ represents the sample_dist value is changed with one unit increase in the value of y . A value of the index can be obtained from a value of an indicator in a bitstream.

[0273] In another embodiment, a value of sample_dist is obtained according to the formula:

$$\text{sample_dist} = ((2 * x) - W + K) * f1(\text{displdxN}) + ((2 * y) - H + K) * f2(\text{displdxS}) - \text{distanceIdx} * \text{stepD} * \text{scaleStep}, \text{ where:}$$

[0274] $f()$ is a function, in an example, function $f()$ is implemented as a lookup table.

[0275] displdxN or displdxS are index values are obtained from a bitstream or are calculated based on a value of an indicator in the bitstream.

[0276] a value of scaleStep is equal to either a value of the width of a coding (decoding or encoding) block or a value of the height of the coding block.

[0277] x and y are the coordinates of the said sample with respect to the top-left sample of the coding block.

[0278] K is an integer offset value. In an example, a value for K is 1.

[0279] In an embodiment, function $f()$ is implemented as a lookup table. Function $f()$ represents an incremental change in the sample_dist value with respect to changing x and y values. In an example, $f(\text{index1})$ represents the sample_dist value is changed with one unit increase in the value of x , whereas the $f(\text{index2})$ represents the sample_dist value is changed with one unit increase in the value of y . A value of the index1 and index2 are index to the table (having an integer value greater than or equal to 0), that can be obtained according to a value of an indicator in a bitstream.

[0280] In an example, implementation of function $f()$ is illustrated in FIG. 14. In this example, a value of the idx is the input parameter (which may be index1 or index2), and the output of the function is illustrated as $f(\text{idx})$. In an example, the $f()$ is an implementation of a cosine function using integer arithmetic, where the idx (input index value) is represents a quantized angle value.

[0281] In one embodiment, a stepD value represents a quantized distance value for the sample distance calculation.

[0282] In one embodiment, as showed in FIG. 13, a stepD value is obtained according to a value of whRatio and a value of angle (α). In an example, a value of stepD can be obtained as

[0283] $\text{stepD} = \text{lookupTable}[\alpha N][\text{whRatio}]$, where a value of αN is an index value that is obtained from the bitstream, or a value of αN is calculated based on a value of an indicator that is obtained from the bitstream. For example, α can be calculated as according to the formula:

$$\begin{aligned}\alpha N &= \text{angleIdx} && \text{if } \text{angleIdx} \geq 0 \ \&\& \ \text{angleIdx} \leq 8, \text{ or} \\ \alpha N &= 16 - \text{angleIdx} && \text{if } \text{angleIdx} > 8 \ \&\& \ \text{angleIdx} \leq 16, \text{ or} \\ \alpha N &= \text{angleIdx} - 16 && \text{if } \text{angleIdx} \geq 16 \ \&\& \ \text{angleIdx} \leq 24, \text{ or} \\ \alpha N &= 32 - \text{angleIdx} && \text{otherwise,}\end{aligned}$$

[0284] where angleIdx is an index value that is obtained according to a bitstream.

[0285] In another example:

$$\begin{aligned}\alpha N &= \text{angleIdx} && \text{if } \text{angleIdx} \geq 0 \ \&\& \ \text{angleIdx} \leq \text{Lim}, \text{ or} \\ \alpha N &= 2 * \text{Lim} - \text{angleIdx} && \text{if } \text{angleIdx} > \text{Lim} \ \&\& \ \text{angleIdx} \leq 2 * \text{Lim}, \text{ or} \\ \alpha N &= \text{angleIdx} - 2 * \text{Lim} && \text{if } \text{angleIdx} > 2 * \text{Lim} \ \&\& \ \text{angleIdx} \leq 3 * \text{Lim}, \text{ or} \\ \alpha N &= 4 * \text{Lim} - \text{angleIdx} && \text{otherwise.}\end{aligned}$$

[0286] Or in another example:

$$\begin{aligned}\alpha N &= \text{angleIdx}, \text{ or} \\ \alpha N &= 2 * \text{Lim} - \text{angleIdx}, \text{ or} \\ \alpha N &= \text{angleIdx} - 2 * \text{Lim}, \text{ or} \\ \alpha N &= 4 * \text{Lim} - \text{angleIdx}.\end{aligned}$$

[0287] In the above formula Lim is a constant predefined value that is related to the number of quantized angle values. For example, if there are 32 different quantized angle values, then the number Lim may be equal to 8 ($4 * Lim$ is equal to 32). In another example Lim can be 4, which corresponds to a total of 16 different quantized angle values.

[0288] In one example, a value of the `sample_dist` is obtained according to $distanceIdx * stepD * scaleStep$, where `distanceIdx` is an index value that is obtained according to a bitstream, and a value of `scaleStep` is obtained according to either a value of width of the block or a value of height of the block. The result of the multiplication represents a distance of the separation line to the center point of the coding block (which has coordinates $x=W/2$ and $y=H/2$).

[0289] In an example, the lookup table that according to is a predefined table. The predefined table has the following advantages:

[0290] Obtaining the distance of a sample to a separation line is usually complex, it requires solving trigonometric equations which is not acceptable when implementing a video coding standard, which targets mass produced consumer products.

[0291] In some embodiments, the sample distance is obtained according to a lookup table (which may be predefined), which pre-calculated intermediate results according to `whRatio` and `alpha`, which are already according to integer arithmetic (hence, in an example, all `stepD` values are integers). It the intermediate result that is obtained using the lookup table is selected carefully since:

[0292] The lookup table comprises the intermediate calculation results for complex operations (trigonometric calculations), therefore implementation complexity is reduced.

[0293] The size of the table is kept small (which would require memory).

[0294] In another example, a value of the `sample_dist` is obtained according to $distanceIdx * (stepD + T) * scaleStep$, where T is an offset value which has an integer value. In an example, a value of T is 32.

[0295] As shown in FIG. 20, a method of coding implemented by a decoding device or an encoding device is disclosed, the method comprises:

[0296] S2001: obtaining a value of an angle parameter for a current block.

[0297] In an example, the value of the angle parameter may be obtained by parsing an index value coded in an video bitstream, or the value of the angle parameter may be determined according to a syntax value which is parsed from the video bitstream.

[0298] The bitstream may be obtained according to wireless network or wired network. The bitstream may be transmitted from a website, server, or other remote source using a coaxial cable, fiber optic cable, twisted pair, digital subscriber line (DSL), or wireless technologies such as infrared, radio, microwave, WIFI, Bluetooth, LTE or 5G.

[0299] In an embodiment, a bitstream is a sequence of bits, e.g. in the form of a network abstraction layer (NAL) unit stream or a byte stream, that forms the representation of a sequence of access units (AUs) forming one or more coded video sequences (CVSs).

[0300] In some embodiments, for a decoding process, a decoder side reads a bitstream and derives decoded pictures from the bitstream; for an encoding process, an encoder side produces a bitstream.

[0301] Normally, a bitstream will comprise syntax elements that are formed by a syntax structure. syntax element: An element of data represented in the bitstream.

[0302] syntax structure: Zero or more syntax elements present together in the bitstream in a specified order.

[0303] In a specific example, bitstream formats specify the relationship between the network abstraction layer (NAL) unit stream and byte stream, either of which are referred to as the bitstream.

[0304] The bitstream can be, for example, in one of two formats: the NAL unit stream format or the byte stream format. The NAL unit stream format is conceptually the more “basic” type. The NAL unit stream format comprises a sequence of syntax structures called NAL units. This sequence is ordered in decoding order. There are constraints imposed on the decoding order (and contents) of the NAL units in the NAL unit stream.

[0305] The byte stream format can be constructed from the NAL unit stream format by ordering the NAL units in decoding order and prefixing each NAL unit with a start code prefix and zero or more zero-valued bytes to form a stream of bytes. The NAL unit stream format can be extracted from the byte stream format by searching for the location of the unique start code prefix pattern within this stream of bytes.

[0306] This clause specifies an embodiment of the relationship between source and decoded pictures that is given via the bitstream.

[0307] The video source that is represented by the bitstream is a sequence of pictures in decoding order.

[0308] In an example, obtaining a value of an indicator for a current block according to the bitstream. The value of the indicator is used to obtain the value of the angle parameter.

[0309] In an embodiment, the value of the indicator is used to specify a partitioning shape of the geometric partitioning merge mode. For example, the indicator may be `merge_gpm_partition_idx[x0][y0]`, wherein `merge_gpm_partition_idx[x0][y0]` specifies the partitioning shape of the geometric partitioning merge mode. The array indices $x0$, $y0$ specify the location ($x0$, $y0$) of the top-left luma sample of the considered coding block relative to the top-left luma sample of the picture.

[0310] Normally, the value of `merge_gpm_partition_idx[x0][y0]` is decoded from a bitstream. In an example, a value range for `merge_gpm_partition_idx[ ][ ]` is from 0 to 63, including 0 and 63. In an example, a decoding process for `merge_gpm_partition_idx[ ][ ]` is “bypass”.

[0311] When `merge_gpm_partition_idx[x0][y0]` is not present, it is inferred to be equal to 0.

[0312] The value of the angle parameter for the current block is obtained according to the value of the indicator and a predefined lookup table.

[0313] In an embodiment, the partition angle variable `angleIdx` (angle parameter) and the distance variable `distanceIdx` of the geometric partitioning mode are set according to the value of `merge_gpm_partition_idx[xCb][yCb]` (indicator) as specified in the following table. In the implementation, this relationship can be implemented according to table 4 or according to a function.

TABLE 4

Specification of angleIdx and distanceIdx based on merge_gpm_partition_idx.	
merge_gpm_partition_idx	
	0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
angleIdx	0 0 2 2 2 2 3 3 3 3 4 4 4 4 5 5
distanceIdx	1 3 0 1 2 3 0 1 2 3 0 1 2 3 0 1
merge_gpm_partition_idx	
	16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
angleIdx	5 5 8 8 11 11 11 11 12 12 12 12 13 13 13 13
distanceIdx	2 3 1 3 0 1 2 3 0 1 2 3 0 1 2 3
merge_gpm_partition_idx	
	32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47
angleIdx	14 14 14 14 16 16 18 18 18 19 19 19 20 20 20 21
distanceIdx	0 1 2 3 1 3 1 2 3 1 2 3 1 2 3 1
merge_gpm_partition_idx	
	48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63
angleIdx	21 21 24 24 27 27 27 28 28 28 29 29 29 30 30 30
distanceIdx	2 3 1 3 1 2 3 1 2 3 1 2 3 1 2 3

[0314] S2002: obtaining a value of a width of the current block and a value of a height of the current block.

[0315] In an example, the W and H are width and height of the current block in number of samples, W and H are parsed from the bitstream. In an example, a coding block with width and height both equal to 8 is a square block that comprises 64 samples. In another example, the W and H are width and height of the current block, in number of luma samples.

[0316] S2003: calculating a ratio value between the value of the width and the value of height.

[0317] In an embodiment, a value of whRatio representing a ratio between the width and height of the current coding block.

[0318] In an example, whRatio=H/W; or whRatio=W/H.

[0319] In another example, two variables nCbW and nCbH specify the width and the height of the current coding block, and a variable cIdx specifies a colour component index.

[0320] Variables nW, nH and whRatio are derived as follows:

[0321] $nW = (cIdx == 0) ? nCbW : nCbW * SubWidthC;$

[0322] $nH = (cIdx == 0) ? nCbH : nCbH * SubHeightC;$

[0323] $whRatio = nH / nW.$

[0324] In an example, the variables SubWidthC and SubHeightC are specified in table 5, depending on the chroma format sampling structure, which is specified through sps_chroma_format_idc and sps_separate_colour_plane_flag.

TABLE 5

SubWidthC and SubHeightC values derived from sps_chroma_format_idc and sps_separate_colour_plane_flag				
sps_chroma_ format_idc	sps_separate_ colour_ plane_flag	Chroma format	SubWidthC	SubHeightC
0	0	Monochrome	1	1
1	0	4:2:0	2	2
2	0	4:2:2	2	1
3	0	4:4:4	1	1
3	1	4:4:4	1	1

[0325] In monochrome sampling there is only one sample array, which is nominally considered the luma array.

[0326] In 4:2:0 sampling, each of the two chroma arrays has half the height and half the width of the luma array.

[0327] In 4:2:2 sampling, each of the two chroma arrays has the same height and half the width of the luma array.

[0328] In 4:4:4 sampling, depending on the value of sps_separate_colour_plane_flag, the following applies:

[0329] If sps_separate_colour_plane_flag is equal to 0, each of the two chroma arrays has the same height and width as the luma array.

[0330] Otherwise (sps_separate_colour_plane_flag is equal to 1), the three colour planes are separately processed as monochrome sampled pictures.

[0331] S2004: calculating a sample distance value for a sample in the current block according to the value of angle parameter and the ratio value.

[0332] In an embodiment, operation S2004 comprises:

[0333] Operation 1: Obtain a shiftHor value according to a lookup table, the value of the alpha and the value of the whRatio.

[0334] In an example, the value of the alpha (alphaN or angleIdx) and the value of the whRatio are used as index

values of the lookup table. The shiftHor value can also be obtained according to a function, wherein the value of the alpha (alphaN or angleIdx) and the value of the whRatio are the input to the function and the shiftHor value is the output of the function. In an example, the result of the function is similar or same as the lookup table. In an example, the shiftHor value represents a quantization step size for the sample distance calculation process.

[0335] In another example, the function may be represented by $\text{shiftHor} = (\text{angleIdx} \% 16 == 8) \parallel (\text{angleIdx} \% 16 != 0 \ \&\& \ \text{whRatio} > 0) ? 0 : 1$.

[0336] Operation 2: a value of the sample_dist is calculated according to the shiftHor value.

[0337] In an example, for calculating sample distance (sample_dist), first, variables offsetX and offsetY are derived as follows:

If shiftHor is equal to 0, the following applies:

$$\begin{aligned} \text{offsetX} &= (-nW) >> 1, \\ \text{offsetY} &= ((-nH) >> 1) + \\ &(\text{angleIdx} < 16 ? (\text{distanceIdx} * nH) >> 3 : -((\text{distanceIdx} * nH) >> 3)); \end{aligned}$$

Otherwise (shiftHor is equal to 1), the following applies:

$$\begin{aligned} \text{offsetX} &= ((-nW) >> 1) + \\ &(\text{angleIdx} < 16 ? (\text{distanceIdx} * nW) >> 3 : -((\text{distanceIdx} * nW) >> 3)); \\ \text{offsetY} &= (-nH) >> 1; \end{aligned}$$

[0338] The variables xL and yL are derived as follows:

$$\begin{aligned} xL &= (\text{cIdx} == 0) ? x : x * \text{SubWidthC} \\ yL &= (\text{cIdx} == 0) ? y : y * \text{SubHeightC} \\ \text{sample_dist} &= (((xL + \text{offsetX}) << 1) + 1) * \text{disLut}[\text{displacementX}] + \\ &(((yL + \text{offsetY}) << 1) + 1) * \text{disLut}[\text{displacementY}]. \end{aligned}$$

[0339] The variables displacementX and displacementY are derived as follows:

- [0340] $\text{hwRatio} = \text{cbHeight} / \text{cbWidth}$;
- [0341] $\text{displacementX} = \text{angleIdx}$;
- [0342] $\text{displacementY} = (\text{angleIdx} + 8) \% 32$.

The array disLut is specified in table 3.

[0343] In some embodiments, according to a geometric model, samples in a coding block are considered as located in two sub-blocks. Sub-block A or sub-block B may comprise a part (but not all) of the samples in the current coding block. Sub-block A or sub-block B may be represented according to the sign of a sample_dist of each samples. The sample_dist may be obtained according to the examples and embodiments in the other paragraphs.

[0344] S2005: obtaining a prediction value for the sample in the current block according to the sample distance value.

[0345] In one implementation, the obtaining a prediction value for the sample, according to the value of the sample distance for the sample, comprises:

[0346] calculating two weighting factors according to the sample distance value; and

[0347] obtaining the prediction value for the sample according to a first prediction value, a second prediction value and the two weighting factors.

[0348] In one implementation, the value of the sample distance represents the horizontal distance or the vertical distance, or a combination of the vertical and horizontal distances, of the said sample to a separation line, wherein the separation line is used to divide a coding block into two sub-blocks.

[0349] In one example, the calculated sample_dist is used to calculate weighting factors, the weighting factors are used for the combination of a first prediction value and a second prediction value corresponding to the said sample. In an example, the weighting factors are denoted as sampleWeight1 and sampleWeight2, referring to the weight corresponding to the first prediction value and the weight corresponding to the second prediction value.

[0350] In one example, weighting factors are calculated according to the following functions,

$$\begin{aligned} \text{weightIdxL} &= \text{partFlip} ? 32 + \text{sample_dist} : 32 - \text{sample_dist}; \\ wValue &= \text{Clip3}(0, 8, (\text{weightIdxL} + 4) >> 3). \end{aligned}$$

[0351] In this example, wValue is sampleWeight1, and 8-wValue is sampleWeight2. The variable partFlip is determined according to a value of angleIdx. In an example, $\text{partFlip} = (\text{angleIdx} >= 13 \ \&\& \ \text{angleIdx} <= 27) ? 0 : 1$, or $\text{partFlip} = (\text{angleIdx} >= 13 \ \&\& \ \text{angleIdx} <= 27) ? 1 : 0$.

[0352] In one example, the combined value of the prediction sample at the sample coordinate (x,y) is calculated according to, the first prediction value at coordinate (x,y), the second prediction value at coordinate (x,y), the sampleWeight1 and the sampleWeight2.

[0353] In an example, the prediction sample value is derived as follows:

$$\begin{aligned} \text{pbSamples}[x][y] &= \\ &\text{Clip3}(0, (1 << \text{BitDepth}) - 1, (\text{predSampledLA}[x][y] * wValue + \\ &\text{predSamplesLB}[x][y] * (8 - wValue) + \text{offset1}) >> \text{shift1}). \end{aligned}$$

Wherein bitDepth represents sample bit depth, variable shift1 is obtained according to the bitDepth, in an example, $\text{shift1} = \text{Max}(5, 17 - \text{BitDepth})$; Variable offset1 is obtained according to the shift1, in an example, $\text{offset1} = 1 << (\text{shift1} - 1)$, predSamplesLA and predSamplesLB are two (nCbw) × (nCbh) arrays.

[0354] As shown in FIG. 21, an apparatus 2100 is disclosed. The apparatus 2100 comprises: parameter obtaining module 2101, which is configured to obtain a value of an angle parameter for a current block; the parameter obtaining module 2101 is also configured to obtain a value of a width of the current block and a value of a height of the current block; ratio calculating module 2102, which is configured to calculate a ratio value between the value of the width and the value of height; sample distance calculating module 2103, which is configured to calculate a sample distance value for a sample in the current block according to the value of angle

parameter and the ratio value; prediction module **2104**, which is configured to obtain a prediction value for the sample in the current block according to the sample distance value.

[0355] The method according to the embodiments of the disclosure can be performed by the apparatus **2100**. Further features and implementation forms of the above methods correspond to the features and implementation forms of the apparatus **2100**.

[0356] Example 1. A method of coding implemented by a decoding device or encoding device, the method comprising:

[0357] obtaining a value of angle parameter for a current block;

[0358] obtaining a value of width of the current block and a value of height of the current block;

[0359] calculating a ratio value between the value of the width and the value of height;

[0360] obtaining a first value according to the value of angle parameter and the ratio value;

[0361] calculating a sample distance value for a sample in the current block according to the first value;

[0362] obtaining a prediction value for the sample in the current block according to the sample distance value.

[0363] Example 2. The method of example 1, wherein the first value is obtained according to a predefined lookup table, the value of angular parameter and the ratio value are used as an index value for the predefined lookup table.

[0364] Example 3. The method of example 2, wherein the predefined lookup table comprises 9 different angular values (0 to 8).

[0365] Example 4. The method of example 2, wherein the predefined lookup table comprises 5 different angular values (0 to 4).

[0366] Example 5. The method of any one of examples 2 to 4, wherein the predefined lookup table comprises 4 different ratio values (0 to 3).

[0367] Example 6. The method of any one of examples 2 to 4, wherein the predefined lookup table comprises 5 different ratio values (0 to 4).

[0368] Example 7. The method of any one of examples 2 to 4, wherein the predefined lookup table comprises 3 different ratio values (0 to 2).

[0369] Example 8. The method of any one of examples 2 to 7, wherein the predefined lookup table comprises a first value corresponding to each combination of angular values and ratio values.

[0370] Example 9. The method of any one of examples 1 to 8, wherein the calculating a sample distance value for a sample in the current block according to the first value, comprises:

[0371] calculating the sample distance value for the sample in the current block according to the first value and the value of the angular parameter.

[0372] Example 10. The method of any one of examples 1 to 9, wherein the ratio value is obtained according to,

$$whRatio = (wIdx \geq hIdx) ? wIdx : hIdx - wIdx,$$

[0373] where whRatio is the ratio value, W is the value of the width, H is the value of the height, a value of wIdx is equal to $\log_2(W)-3$, and a value of hIdx is equal to $\log_2(H)-3$.

[0374] Example 11. The method of any one of examples 1 to 9, wherein the ratio value is obtained according to,

$$whRatio = (W \geq H) ? W/H : H/W,$$

[0375] wherein whRatio is the ratio value, W is the value of the width, H is the value of the height.

[0376] Example 12. The method of any one of examples 1 to 11, wherein the sample distance value is calculated according to,

$$\text{sample_dist} = ((2^x) - W + K) * f1(\alpha) + ((2^y) - H + K) * f2(\alpha) - \text{distanceIdx} * \text{stepD} * \text{scaleStep},$$

where sample_dist is the sample distance value, f1() and f2() are functions, and alpha is the value angular parameter (In an example, f1() and f2() are implemented as lookup tables, a value of the alpha is an index value for the lookup tables); a value of scaleStep is equal to either the value of the width or the value of the height; x and y are the coordinates of the sample with respect to the top-left sample of the current block; and K is an integer offset value.

[0377] Example 13. The method of any one of examples 1 to 11, wherein the sample distance value is calculated according to,

$$\text{sample_dist} = ((2^x) - W + K) * f(\text{displDxN}) + ((2^y) - H + K) * f(\text{displDxS}) - \text{distanceIdx} * \text{stepD} * \text{scaleStep},$$

where sample_dist is the sample distance value, f() is a function (in an example, function f() is implemented as a lookup table), displDxN or displDxS are index values (in an example, displDxN or displDxS are obtained from a bitstream or are calculated based on a value of an indicator in the bitstream), a value of scaleStep is equal to either the value of the width or the value of the height, x and y are the coordinates of the sample with respect to the top-left sample of the current block, K is an integer offset value (In an example, a value for K is 1).

[0378] Example 14. A decoder (**30**) or an encoder comprising processing circuitry for carrying out the method according to any one of examples 1 to 13.

[0379] Example 15. A computer program product comprising a program code for performing the method according to any one of examples 1 to 13.

[0380] Example 16. A decoder or an encoder, comprising:

[0381] one or more processors; and

[0382] a non-transitory computer-readable storage medium coupled to the processors and storing programming for execution by the processors, wherein the programming, when executed by the processors, configures the decoder to carry out the method according to any one of examples 1 to 13.

[0383] Following is an explanation of the applications of the encoding method as well as the decoding method as shown in the above-mentioned embodiments, and a system using them.

[0384] FIG. **18** is a block diagram showing a content supply system **3100** for realizing content distribution service. This content supply system **3100** includes capture device **3102**, terminal device **3106**, and optionally includes

display **3126**. The capture device **3102** communicates with the terminal device **3106** over communication link **3104**. The communication link may include the communication channel **13** described above. The communication link **3104** includes but not limited to WIFI, Ethernet, Cable, wireless (3G/4G/5G), USB, or any kind of combination thereof, or the like.

[0385] The capture device **3102** generates data, and may encode the data by the encoding method as shown in the above embodiments. Alternatively, the capture device **3102** may distribute the data to a streaming server (not shown in the Figures), and the server encodes the data and transmits the encoded data to the terminal device **3106**. The capture device **3102** includes but not limited to camera, smart phone or Pad, computer or laptop, video conference system, PDA, vehicle mounted device, or a combination of any of them, or the like. For example, the capture device **3102** may include the source device **12** as described above. When the data includes video, the video encoder **20** included in the capture device **3102** may actually perform video encoding processing. When the data includes audio (i.e., voice), an audio encoder included in the capture device **3102** may actually perform audio encoding processing. For some practical scenarios, the capture device **3102** distributes the encoded video and audio data by multiplexing them together. For other practical scenarios, for example in the video conference system, the encoded audio data and the encoded video data are not multiplexed. Capture device **3102** distributes the encoded audio data and the encoded video data to the terminal device **3106** separately.

[0386] In the content supply system **3100**, the terminal device **310** receives and reproduces the encoded data. The terminal device **3106** could be a device with data receiving and recovering capability, such as smart phone or Pad **3108**, computer or laptop **3110**, network video recorder (NVR)/digital video recorder (DVR) **3112**, TV **3114**, set top box (STB) **3116**, video conference system **3118**, video surveillance system **3120**, personal digital assistant (PDA) **3122**, vehicle mounted device **3124**, or a combination of any of them, or the like capable of decoding the above-mentioned encoded data. For example, the terminal device **3106** may include the destination device **14** as described above. When the encoded data includes video, the video decoder **30** included in the terminal device is prioritized to perform video decoding. When the encoded data includes audio, an audio decoder included in the terminal device is prioritized to perform audio decoding processing.

[0387] For a terminal device with its display, for example, smart phone or Pad **3108**, computer or laptop **3110**, network video recorder (NVR)/digital video recorder (DVR) **3112**, TV **3114**, personal digital assistant (PDA) **3122**, or vehicle mounted device **3124**, the terminal device can feed the decoded data to its display. For a terminal device equipped with no display, for example, STB **3116**, video conference system **3118**, or video surveillance system **3120**, an external display **3126** is contacted therein to receive and show the decoded data.

[0388] When each device in this system performs encoding or decoding, the picture encoding device or the picture decoding device, as shown in the above-mentioned embodiments, can be used.

[0389] FIG. 19 is a diagram showing a structure of an example of the terminal device **3106**. After the terminal device **3106** receives stream from the capture device **3102**,

the protocol proceeding unit **3202** analyzes the transmission protocol of the stream. The protocol includes but not limited to Real Time Streaming Protocol (RTSP), Hyper Text Transfer Protocol (HTTP), HTTP Live streaming protocol (HLS), MPEG-DASH, Real-time Transport protocol (RTP), Real Time Messaging Protocol (RTMP), or any kind of combination thereof, or the like.

[0390] After the protocol proceeding unit **3202** processes the stream, stream file is generated. The file is outputted to a demultiplexing unit **3204**. The demultiplexing unit **3204** can separate the multiplexed data into the encoded audio data and the encoded video data. As described above, for some practical scenarios, for example in the video conference system, the encoded audio data and the encoded video data are not multiplexed. In this situation, the encoded data is transmitted to video decoder **3206** and audio decoder **3208** without through the demultiplexing unit **3204**.

[0391] Via the demultiplexing processing, video elementary stream(ES), audio ES, and optionally subtitle are generated. The video decoder **3206**, which includes the video decoder **30** as explained in the above mentioned embodiments, decodes the video ES by the decoding method as shown in the above-mentioned embodiments to generate video frame, and feeds this data to the synchronous unit **3212**. The audio decoder **3208**, decodes the audio ES to generate audio frame, and feeds this data to the synchronous unit **3212**. Alternatively, the video frame may store in a buffer (not shown in FIG. 19) before feeding it to the synchronous unit **3212**. Similarly, the audio frame may store in a buffer (not shown in FIG. 19) before feeding it to the synchronous unit **3212**.

[0392] The synchronous unit **3212** synchronizes the video frame and the audio frame, and supplies the video/audio to a video/audio display **3214**. For example, the synchronous unit **3212** synchronizes the presentation of the video and audio information. Information may code in the syntax using time stamps concerning the presentation of coded audio and visual data and time stamps concerning the delivery of the data stream itself.

[0393] If subtitle is included in the stream, the subtitle decoder **3210** decodes the subtitle, and synchronizes it with the video frame and the audio frame, and supplies the video/audio/subtitle to a video/audio/subtitle display **3216**.

[0394] The present disclosure is not limited to the above-mentioned system, and either the picture encoding device or the picture decoding device in the above-mentioned embodiments can be incorporated into other system, for example, a car system.

Mathematical Operators

[0395] The mathematical operators used in this disclosure are similar to those used in the C programming language. However, the results of integer division and arithmetic shift operations are defined more precisely, and additional operations are defined, such as exponentiation and real-valued division. Numbering and counting conventions generally begin from 0, e.g., "the first" is equivalent to the 0-th, "the second" is equivalent to the 1-th, etc.

Arithmetic Operators

[0396] The following arithmetic operators are defined as follows:

- [0397] + Addition
- [0398] − Subtraction (as a two-argument operator) or negation (as a unary prefix operator)
- [0399] * Multiplication, including matrix multiplication
- [0400] x^y Exponentiation. Specifies x to the power of y . In other contexts, such notation is used for superscripting not intended for interpretation as exponentiation.
- [0401] / Integer division with truncation of the result toward zero. For example, $7/4$ and $-7/-4$ are truncated to 1 and $-7/4$ and $7/-4$ are truncated to -1 .
- [0402] ÷ Used to denote division in mathematical equations where no truncation or rounding is intended.

$$\frac{x}{y}$$

Used to denote division in mathematical equations where no truncation or rounding is intended.

$$\sum_{i=x}^y$$

f(i) The summation of f(i) with i taking all integer values from x up to and including y.

- [0403] $x \% y$ Modulus. Remainder of x divided by y , defined only for integers x and y with $x \geq 0$ and $y > 0$.

Logical Operators

[0404] The following logical operators are defined as follows:

- [0405] $x \&\& y$ Boolean logical “and” of x and y
- [0406] $x||y$ Boolean logical “or” of x and y
- [0407] ! Boolean logical “not”
- [0408] $x?y:z$ If x is TRUE or not equal to 0, evaluates to the value of y ; otherwise, evaluates to the value of z .

Relational Operators

[0409] The following relational operators are defined as follows:

- [0410] > Greater than
- [0411] >= Greater than or equal to
- [0412] < Less than
- [0413] <= Less than or equal to
- [0414] == Equal to
- [0415] != Not equal to

[0416] When a relational operator is applied to a syntax element or variable that has been assigned the value “na” (not applicable), the value “na” is treated as a distinct value for the syntax element or variable. The value “na” is considered not to be equal to any other value.

Bit-Wise Operators

[0417] The following bit-wise operators are defined as follows:

- [0418] & Bit-wise “and”. When operating on integer arguments, operates on a two’s complement representation of the integer value. When operating on a binary

argument that contains fewer bits than another argument, the shorter argument is extended by adding more significant bits equal to 0.

- [0419] | Bit-wise “or”. When operating on integer arguments, operates on a two’s complement representation of the integer value. When operating on a binary argument that contains fewer bits than another argument, the shorter argument is extended by adding more significant bits equal to 0.
- [0420] ^ Bit-wise “exclusive or”. When operating on integer arguments, operates on a two’s complement representation of the integer value. When operating on a binary argument that contains fewer bits than another argument, the shorter argument is extended by adding more significant bits equal to 0.
- [0421] $x \gg y$ Arithmetic right shift of a two’s complement integer representation of x by y binary digits. This function is defined only for non-negative integer values of y . Bits shifted into the most significant bits (MSBs) as a result of the right shift have a value equal to the MSB of x prior to the shift operation.
- [0422] $x \ll y$ Arithmetic left shift of a two’s complement integer representation of x by y binary digits. This function is defined only for non-negative integer values of y . Bits shifted into the least significant bits (LSBs) as a result of the left shift have a value equal to 0.

Assignment Operators

[0423] The following arithmetic operators are defined as follows:

- [0424] = Assignment operator
- [0425] ++ Increment, i.e., $x++$ is equivalent to $x=x+1$; when used in an array index, evaluates to the value of the variable prior to the increment operation.
- [0426] — Decrement, i.e., $x--$ is equivalent to $x=x-1$; when used in an array index, evaluates to the value of the variable prior to the decrement operation.
- [0427] += Increment by amount specified, i.e., $x+=3$ is equivalent to $x=x+3$, and $x+=(-3)$ is equivalent to $x=x+(-3)$.
- [0428] -= Decrement by amount specified, i.e., $x-=3$ is equivalent to $x=x-3$, and $x-=(-3)$ is equivalent to $x=x-(-3)$.

Range Notation

[0429] The following notation is used to specify a range of values:

- [0430] $x=y \dots z$ x takes on integer values starting from y to z , inclusive, with x , y , and z being integer numbers and z being greater than y .

Mathematical Functions

[0431] The following mathematical functions are defined:

$$\text{Abs}(x) = \begin{cases} x & ; x \geq 0 \\ -x & ; x < 0 \end{cases}$$

- [0432] Asin (x) the trigonometric inverse sine function, operating on an argument x that is in the range of -1.0 to 1.0 , inclusive, with an output value in the range of $-\pi/2$ to $\pi/2$, inclusive, in units of radians
- [0433] Atan (x) the trigonometric inverse tangent function, operating on an argument x , with an output value in the range of $-\pi/2$ to $\pi/2$, inclusive, in units of radians

$$\text{Atan2}(y, x) = \begin{cases} \text{Atan}\left(\frac{y}{x}\right) & ; \quad x > 0 \\ \text{Atan}\left(\frac{y}{x}\right) + \pi & ; \quad x < 0 \ \&\& \ y \geq 0 \\ \text{Atan}\left(\frac{y}{x}\right) - \pi & ; \quad x < 0 \ \&\& \ y < 0 \\ +\frac{\pi}{2} & ; \quad x == 0 \ \&\& \ y \geq 0 \\ -\frac{\pi}{2} & ; \quad \text{otherwise} \end{cases}$$

[0434] Ceil(x) the smallest integer greater than or equal to x.

[0435] Clip1_y(x)=Clip3(0, (1<<BitDepth_y)-1, x)

[0436] Clip1_c(x)=Clip3(0, (1<<BitDepth_c)-1, x)

$$\text{Clip3}(x, y, z) = \begin{cases} x & ; \quad z < x \\ y & ; \quad z > y \\ z & ; \quad \text{otherwise} \end{cases}$$

[0437] Cos(x) the trigonometric cosine function operating on an argument x in units of radians.

[0438] Floor(x) the largest integer less than or equal to x.

$$\text{GetCurrMsb}(a, b, c, d) = \begin{cases} c + d & ; \quad b - a \geq d/2 \\ c - d & ; \quad a - b > d/2 \\ c & ; \quad \text{otherwise} \end{cases}$$

[0439] Ln(x) the natural logarithm of x (the base-e logarithm, where e is the natural logarithm base constant 2.718 281 828 . . .).

[0440] Log₂(x) the base-2 logarithm of x.

[0441] Log₁₀(x) the base-10 logarithm of x.

$$\text{Min}(x, y) = \begin{cases} x & ; \quad x \leq y \\ y & ; \quad x > y \end{cases}$$

$$\text{Max}(x, y) = \begin{cases} x & ; \quad x \geq y \\ y & ; \quad x < y \end{cases}$$

[0442] Round(x)=Sign(x)*Floor(Abs(x)+0.5)

$$\text{Sign}(x) = \begin{cases} 1 & ; \quad x > 0 \\ 0 & ; \quad x == 0 \\ -1 & ; \quad x < 0 \end{cases}$$

[0443] Sin(x) the trigonometric sine function operating on an argument x in units of radians

$$\text{Sqrt}(x) = \sqrt{x}$$

$$\text{Swap}(x, y) = (y, x)$$

[0444] Tan(x) the trigonometric tangent function operating on an argument x in units of radians

Order of Operation Precedence

[0445] When an order of precedence in an expression is not indicated explicitly by use of parentheses, the following rules apply:

[0446] Operations of a higher precedence are evaluated before any operation of a lower precedence.

[0447] Operations of the same precedence are evaluated sequentially from left to right.

[0448] Table 6 below specifies the precedence of operations from highest to lowest; a higher position in the table indicates a higher precedence.

[0449] For those operators that are also used in the C programming language, the order of precedence used in this Specification is the same as used in the C programming language.

TABLE 6

Operation precedence from highest (at top of table) to lowest (at bottom of table) operations (with operands x, y, and z)
"x++", "x--"
"!x", "-x" (as a unary prefix operator)
x ²
"x * y", "x / y", "x ÷ y", $\frac{x}{y}$, "x % y"
"x + y", "x - y" (as a two-argument operator),
$\sum_{i=x}^y f(i)$
"x << y", "x >> y"
"x < y", "x <= y", "x > y", "x >= y"
"x = y", "x != y"
"x & y"
"x y"
"x && y"
"x y"
"x ? y : z"
"x . . . y"
"x = y", "x += y", "x -= y"

Text Description of Logical Operations

[0450] In the text, a statement of logical operations as would be described mathematically in the following form:

```

if( condition 0 )
    statement 0
else if( condition 1 )
    statement 1
. . .
else /* informative remark on remaining condition */
    statement n

```

-continued

may be described in the following manner:

... as follows / ... the following applies:

If condition 0, statement 0

Otherwise, if condition 1, statement 1

...

Otherwise (informative remark on remaining condition), statement n

[0451] Each “If ... Otherwise, if ... Otherwise, ...” statement in the text is introduced with “... as follows” or “... the following applies” immediately followed by “If ...”. The last condition of the “If ... Otherwise, if ... Otherwise, ...” is always an “Otherwise, ...”. Interleaved “If ... Otherwise, if ... Otherwise, ...” statements can be identified by matching “... as follows” or “... the following applies” with the ending “Otherwise, ...”.

[0452] In the text, a statement of logical operations as would be described mathematically in the following form:

if(condition 0a && condition 0b)

statement 0

else if(condition 1a || condition 1b)

statement 1

...

else

statement n

may be described in the following manner:

... as follows / ... the following applies:

If all of the following conditions are true, statement 0:

condition 0a

condition 0b

Otherwise, if one or more of the following conditions are true, statement 1:

condition 1a

condition 1b

...

Otherwise, statement n

[0455] Embodiments, e.g. of the encoder **20** and the decoder **30**, and functions described herein, e.g. with reference to the encoder **20** and the decoder **30**, may be implemented in hardware, software, firmware, or any combination thereof. If implemented in software, the functions may be stored on a computer-readable medium or transmitted over communication media as one or more instructions or code and executed by a hardware-based processing unit. Computer-readable media may include computer-readable storage media, which corresponds to a tangible medium such as data storage media, or communication media including any

[0453] In the text, a statement of logical operations as would be described mathematically in the following form:

if(condition 0)

statement 0

if(condition 1)

statement 1

may be described in the following manner:

When condition 0, statement 0

When condition 1, statement 1.

[0454] Although embodiments of the disclosure have been primarily described based on video coding, it should be noted that embodiments of the coding system **10**, encoder **20** and decoder **30** (and correspondingly the system **10**) and the other embodiments described herein may also be configured for still picture processing or coding, i.e. the processing or coding of an individual picture independent of any preceding or consecutive picture as in video coding. In general only inter-prediction units **244** (encoder) and **344** (decoder) may not be available in case the picture processing coding is limited to a single picture **17**. All other functionalities (also referred to as tools or technologies) of the video encoder **20** and video decoder **30** may equally be used for still picture processing, e.g. residual calculation **204/304**, transform **206**, quantization **208**, inverse quantization **210/310**, (inverse) transform **212/312**, partitioning **262/362**, intra-prediction **254/354**, and/or loop filtering **220**, **320**, and entropy coding **270** and entropy decoding **304**.

medium that facilitates transfer of a computer program from one place to another, e.g., according to a communication protocol. In this manner, computer-readable media generally may correspond to (1) tangible computer-readable storage media which is non-transitory or (2) a communication medium such as a signal or carrier wave. Data storage media may be any available media that can be accessed by one or more computers or one or more processors to retrieve instructions, code and/or data structures for implementation of the techniques described in this disclosure. A computer program product may include a computer-readable medium.

[0456] By way of example, and not limiting, such computer-readable storage media can comprise RAM, ROM, EEPROM, CD-ROM or other optical disk storage, magnetic disk storage, or other magnetic storage devices, flash memory, or any other medium that can be used to store desired program code in the form of instructions or data structures and that can be accessed by a computer. Also, any connection is properly termed a computer-readable medium. For example, if instructions are transmitted from a website, server, or other remote source using a coaxial cable, fiber optic cable, twisted pair, digital subscriber line (DSL), or wireless technologies such as infrared, radio, and microwave, then the coaxial cable, fiber optic cable, twisted pair, DSL, or wireless technologies such as infrared, radio, and microwave are included in the definition of medium. It should be understood, however, that computer-readable storage media and data storage media do not include connections, carrier waves, signals, or other transitory media, but are instead directed to non-transitory, tangible storage media. Disk and disc, as used herein, includes compact disc (CD), laser disc, optical disc, digital versatile disc (DVD), floppy disk and Blu-ray disc, where disks usually reproduce

data magnetically, while discs reproduce data optically with lasers. Combinations of the above should also be included within the scope of computer-readable media.

[0457] Instructions may be executed by one or more processors, such as one or more digital signal processors (DSPs), general purpose microprocessors, application specific integrated circuits (ASICs), field programmable logic arrays (FPGAs), or other equivalent integrated or discrete logic circuitry. Accordingly, the term “processor,” as used herein may refer to any of the foregoing structure or any other structure suitable for implementation of the techniques described herein. In addition, in some aspects, the functionality described herein may be provided within dedicated hardware and/or software modules configured for encoding and decoding, or incorporated in a combined codec. Also, the techniques could be fully implemented in one or more circuits or logic elements.

[0458] The techniques of this disclosure may be implemented in a wide variety of devices or apparatuses, including a wireless handset, an integrated circuit (IC) or a set of ICs (e.g., a chip set). Various components, modules, or units are described in this disclosure to emphasize functional aspects of devices configured to perform the disclosed techniques, but do not necessarily require realization by

5. The method of claim 1, wherein the value of the angle parameter is an integer value, and a value range for the value of the angle parameter is between 0 to 31, including 0 and 31.

6. The method of claim 1, wherein the comparison value between the value of the width of the current block and the value of the height of the current block is true, when the value of the height of the current block is greater or equal than the value of the width of the current block.

7. The method of claim 1, wherein the first value is equal to either 0 or 1.

8. The method of claim 1, wherein when the first value is equal to 0,

$$\text{offsetX} = (-W) \gg 1, \text{ and } \text{offsetY} = ((-H) \gg 1) + (\text{angleIdx} < 16 ? (\text{distanceIdx} * H) \gg 3 : -((\text{distanceIdx} * H) \gg 3)),$$
 wherein the angleIdx is the angle parameter, and distanceIdx is the value of the distance index, wherein W is the value of the width of the current block, H is the value of the height of the current block, angleIdx is a quantized angle value, and distanceIdx is a quantized distance value.

9. The method of claim 1, wherein when the first value is equal to 1,

$$\begin{aligned} \text{offsetX} &= ((-W) \gg 1) + \\ &(\text{angleIdx} < 16 ? (\text{distanceIdx} * W) > 3 : -((\text{distanceIdx} * W) > 3)), \\ \text{and } \text{offsetY} &= ((-H) \gg 1). \end{aligned}$$

different hardware units. Rather, as described above, various units may be combined in a codec hardware unit or provided by a collection of inter operative hardware units, including one or more processors as described above, in conjunction with suitable software and/or firmware.

1. A method, comprising:

obtaining a value of an angle parameter for a current block, a value of a distance index, a value of a width of the current block, a value of a height of the current block, a first value according to the value of the angle parameter, and a comparison value between the value of the width of the current block and the value of the height of the current block;

calculating a sample distance value for a sample in the current block according to an offsetX value and an offsetY value, wherein the offsetX and offsetY values are obtained according to the first value, the value of the angle parameter, the value of the distance index; and

obtaining a prediction value for the sample in the current block according to the sample distance value.

2. The method of claim 1, wherein the first value represents a quantization step size for the sample distance value calculation.

3. The method of claim 1, wherein the sample distance value represents a horizontal distance or a vertical distance, or a combination of the vertical and horizontal distances, of the sample to a separation line, wherein the separation line divides a coding block into two sub-blocks.

4. The method of claim 1, wherein obtaining the prediction value for the sample in the current block based on the sample distance value comprises:

calculating two weighting factors based on the sample distance value;

obtaining the prediction value for the sample in the current block based on a first prediction value, a second prediction value and the two weighting factors.

10. A video coder, comprising:

one or more processors; and

a computer-readable storage medium coupled to the processors and storing programming for execution by the processors, wherein the programming, when executed by the processors, configures the coder to carry out operations of:

obtaining a value of an angle parameter for a current block, a value of a distance index, a value of a width of the current block, a value of a height of the current block, a first value according to the value of the angle parameter, and a comparison value between the value of the width of the current block and the value of the height of the current block;

calculating a sample distance value for a sample in the current block according to an offsetX value and an offsetY value, wherein the offsetX and offsetY values are obtained according to the first value, the value of the angle parameter, the value of the distance index; and

obtaining a prediction value for the sample in the current block according to the sample distance value.

11. The coder of claim 10, wherein the first value represents a quantization step size for the sample distance value calculation.

12. The coder of claim 10, wherein the sample distance value represents a horizontal distance or a vertical distance, or a combination of the vertical and horizontal distances, of the sample to a separation line, wherein the separation line is used to divide a coding block into two sub-blocks.

13. The coder of claim 10, wherein obtaining the prediction value for the sample in the current block based on the sample distance value comprises:

calculating two weighting factors based on the sample distance value;

obtaining the prediction value for the sample in the current block based on a first prediction value, a second prediction value and the two weighting factors.

14. The coder of claim **10**, wherein the value of the angle parameter is an integer value, and a value range for the value of the angle parameter is between 0 to 31, including 0 and 31.

15. The coder of claim **10**, wherein the comparison value between the value of the width of the current block and the value of the height of the current block is true, when the value of the height of the current block is greater or equal than the value of the width of the current block.

16. The coder of claim **10**, wherein the first value is equal to either 0 or 1.

17. The coder of claim **10**, wherein when the first value is equal to 0,

$\text{offsetX} = (-W) \gg 1$, and $\text{offsetY} = ((-H) \gg 1) + (\text{angleIdx} < 16 ? (\text{distanceIdx} * H) \gg 3 : -((\text{distanceIdx} * H) \gg 3))$, wherein the angleIdx is the angle parameter, and distanceIdx is the value of the distance index, wherein W is the value of the width of the current block, H is the value of the height of the current block, angleIdx is a quantized angle value, and distanceIdx is a quantized distance value.

18. The coder of claim **10**, wherein when the first value is equal to 1,

$\text{offsetX} = ((-W) \gg 1) + (\text{angleIdx} < 16 ? (\text{distanceIdx} * W) > 3 : -((\text{distanceIdx} * W) > 3))$, and $\text{offsetY} = ((-H) \gg 1)$.

19. A non-transitory computer-readable storage medium storing a bitstream and one or more instructions executable by at least one processor to perform operations of encoding or decoding of the bitstream, the operations comprising:

obtain a value of an angle parameter for a current block, a value of a distance index, a value of a width of the current block, a value of a height of the current block, a first value according to the value of the angle parameter, and a comparison value between the value of the width of the current block and the value of the height of the current block;

calculate a sample distance value for a sample in the current block according to an offsetX value and an

offsetY value, wherein the offsetX and offsetY values are obtained according to the first value, the value of the angle parameter, the value of the distance index; and

obtain a prediction value for the sample in the current block according to the sample distance value.

20. The non-transitory computer-readable storage medium of claim **16**, wherein the comparison value between the value of the width of the current block and the value of the height of the current block is true, when the value of the height of the current block is greater or equal than the value of the width of the current block.

* * * * *